

Much Ado about Prompting: LLM Classification of Text Messages from Experiments

Can Çelebi*¹ and Stefan P. Penczynski²

¹*Vienna Center for Experimental Economics (VCEE), University of Vienna,
Oskar-Morgenstern-Platz 1, 1090 Vienna, Austria. Email: can.celebi@univie.ac.at*

²*School of Economics and Centre for Behavioural and Experimental Social Science (CBESS),
University of East Anglia, Norwich Research Park, Norwich NR4 7TJ, United Kingdom.
Email: s.penczynski@uea.ac.uk, Tel. +44 1603 59 1796*

June 16, 2026

Researchers who classify text with large language models often possess codebooks written for human annotators. We ask how these codebooks can serve as prompts. Using three codebooks from experimental economics (one on promise classification, two on strategic thinking), we vary the level of information in the prompt, then vary its formatting, framing and wording at the lowest and highest information levels, across two proprietary and two open-weight models. Used as prompts, the codebooks reach 82–88% agreement with human annotators across the three tasks. On the recognition-heavy task (promise classification), model choice accounts for most of the variation in accuracy; on the learning-heavy tasks (strategic thinking classification), the level of detail in the classification instructions carries comparable weight. These information components partly substitute for one another, whereas model reasoning does not reliably compensate for missing content and yields little or no improvement once the content is present. Larger models make better use of additional information and are more robust to formatting, framing, and wording of the prompt, while smaller models can be hurt by extra information and are more sensitive to how the information is presented. Our results advocate for a shift in focus from prompt engineering techniques (formatting, framing, reasoning, etc.) to the content of the prompt: preparing instructions as one would for human annotators, with detailed context, category definitions, and examples.

*Corresponding author.

1. Introduction

In text classification tasks, researchers require a *codebook*, either written from scratch or adapted for a new setting from an existing one¹ to instruct *human* annotators. These codebooks lay out the underlying context and theory for the classifications, and provide detailed category definitions and annotated examples for each category. When researchers turn to large language models (LLMs) to perform the same task, a natural question arises: can the codebook prepared for human annotators be used directly as an LLM prompt?

The literature offers guidance on how to prompt “effectively”. Researchers are advised to structure their prompts with markdown headers and bullet lists (Han et al., 2025; Törnberg, 2024; Mishra et al., 2021b), to assign role personas (Dunivin, 2025; Carlson and Burbano, 2026; Kong et al., 2023), to provide background context and label descriptions (White et al., 2023; Peskine et al., 2023), to include worked examples (Brown et al., 2020), and to restructure the codebook for the model (Stuhler et al., 2025). However the majority of this advice draws on early prompting research on GPT-2 and GPT-3 (Reynolds and McDonell, 2021; Mishra et al., 2021a), where models were less capable at following instructions and prompts were typically short, one- or two-sentence task descriptions rather than detailed codebooks (Pecher et al., 2026). Following these recommendations requires effort to transform the codebook into something “optimised” for the model, yet whether they still apply to today’s frontier models, or to substantially more detailed prompts, is unclear.

Apart from the informational content of a prompt, there is also the question of how that content is presented: choices about formatting (markdown vs. plain text, bullet lists vs. prose), framing (role persona, task preamble), and phrasing (specific word choices, sentence structure). We refer to these collectively as *surface variations*: changes that alter how the content is presented without changing the informational detail of what it communicates about the classification task. Ideally, classification outcomes should be robust to surface variations. Yet, several studies have shown that LLM outputs can be sensitive to them, producing significant variance in classification outcomes (Sclar et al., 2024; Reiss, 2023). We refer to this sensitivity as *prompt brittleness*: the dependence of classification outcomes on surface variations (Li et al., 2025). If models are brittle in this way, then any particular prompt formulation may not be robust, and iterating on phrasing and format may not lead to a reliably optimal prompting solution.

In this paper we take three classification codebooks from experimental economics

¹For example, the promise classification codebook developed by Charness and Dufwenberg (2006) for a trust game was later adapted by Arad et al. (2024) for a public goods game.

and decompose each into its distinct informational components: classification instructions, experiment context, theoretical background, and worked examples. We then independently vary the level of informational detail in each component in a factorial design and compare classification performance within and across two proprietary (GPT 5.4, GPT 5.4-nano) and two open-weight models (Qwen 3.5 397B, Qwen 3.5 9B). At the lowest and highest levels of informational detail, we additionally investigate the effect of surface variations (formatting, framing, wording) as well as the use of model reasoning on performance across the four models.

Our central finding is that a codebook transcribed into a prompt with minimal modification reaches 82–88% agreement with humans across the four models tested. Performance² is driven primarily by the level of informational detail in the classification instructions: experimental context, theory, category definitions, and examples. These components partly compensate for one another when some or all others are missing, and partly complement one another when combined. This validates the use of detailed codebooks prepared for human annotators as prompts for LLMs.

The benefits of using a codebook as a prompt on model performance, however, depend on both the task and the model. Strategic thinking classification (a learning-heavy task) benefits substantially from detailed instructions, whereas promise classification (a recognition-heavy task) benefits more from model size than from prompt content. Larger models, GPT 5.4 and Qwen 3.5 397B, make better use of additional detail (full codebook content), whereas for smaller models it provides no significant benefit, and can even hurt performance.

Surface variations have no significant effect for GPT 5.4 and only marginal effects for GPT 5.4-nano, while the open-weight Qwen models display some degree of prompt brittleness. However, even in cases where surface variations significantly affect performance, their effects have no systematic direction: the same formatting, framing, or wording change can improve performance in one case and reduce it in another. These surface variations are therefore either inconsequential or unreliable as tools for improving performance.

Enabling model reasoning, or increasing its degree gradually from none to extremely high in the GPT models, does not reliably improve performance. Reasoning is at best a partial substitute when the prompt lacks the informational detail of a codebook, and it is not a complement once detailed context, theory, category definitions, and examples are provided. Furthermore, for GPT models, at high reasoning settings (high and

²Throughout the paper, *performance* (or *accuracy*) refers to the agreement rate between the model’s classification and the ground truth, defined as the label on which two independent human annotators agreed. See Section 4 for details.

extremely high), reasoning either provides no benefit or significantly worsens performance relative to the no reasoning baseline.

These results contribute to the LLM text classification literature in three ways. First, we show that once content unrelated to the classification task is filtered out, an existing codebook prepared for human annotators can be used as-is to serve as a reliable prompt for LLMs. Second, we are the first to jointly examine three dimensions of LLM text classification: informational detail of the prompt, prompt engineering (formatting, framing, phrasing, and reasoning), and model scale. We find that informational detail dictates whether surface variations matter, while model scale dictates how much they matter. Lastly, this joint perspective reconciles conflicting prompting recommendations and explains the documented issue of prompt brittleness in the literature. Our results indicate that prompt brittleness is likely an artifact of classification done with underspecified prompts or relatively small-scale models (e.g., Qwen 9B).

The practical recommendation is to pair a frontier model with a well-defined codebook. By well-defined, we mean a codebook that provides the level of information a researcher would need to give human annotators: the relevant context, underlying theory, category definitions, and examples. Researchers can then spend their effort on getting this informational detail right, rather than on prompt engineering techniques.

2. Related Work

LLMs are increasingly used for text classification in the social sciences, with recent work confirming that LLMs can achieve inter-rater reliability comparable to trained coders across multiple disciplines (Gilardi et al., 2023; Ziems et al., 2024; Ornstein et al., 2025; Chae and Davidson, 2025). As adoption grows, so does the question of how best to instruct the model. Studies testing this question produce apparently conflicting recommendations: formatting matters substantially (Sclar et al., 2024; Han et al., 2025); detailed codebooks help (Lupo et al., 2024; Halterman and Keith, 2025; Atreja et al., 2025); detailed codebooks can hurt (Ruckdeschel, 2025, for small models); personas are recommended (Törnberg, 2024; Dunivin, 2025); personas are ineffective or harmful (Zheng et al., 2024; Kim et al., 2024; Hu et al., 2026; McLaren et al., 2026).

A useful organising principle comes from Pecher et al. (2026), who distinguish *underspecified* prompts — which provide little task description and no label constraints — from *well-specified* prompts that include explicit instructions.³ On underspecified

³Pecher et al. establish this distinction at the lower end of the specification range: their underspecified prompts are essentially raw completion cues of around five words (e.g., “{text}\nSentiment:”), while their well-specified prompts are brief task descriptions of around 25 words (e.g., “Determine

prompts, model predictions are near-chance and any surface variation can shift outcomes; on well-specified prompts, predictions are robust and surface variations become noise.

This distinction reconciles both the conflicting recommendations regarding optimal prompt design (Törnberg, 2024; Zheng et al., 2024; Lupo et al., 2024; Ruckdeschel, 2025) and the broader phenomenon of prompt brittleness (Sclar et al., 2024; Lu et al., 2021; Reiss, 2023; Barrie et al., 2025). Studies reporting significant performance fluctuations due to surface variations use underspecified prompts, models at smaller scales, or both. Although this framework may not resolve all conflicting findings in the literature, it aligns closely with the empirical results most comparable to our own.⁴

3. Research Questions

In our setting, a codebook contains four functional components: classification instructions defining each category, experiment context describing the game mechanics, theory context laying out the formal theoretical framework, and worked examples. This structure is characteristic of experimental economics, where classifications are often grounded in a formal model. Codebooks in other domains may omit the theory layer, but classification instructions and examples are likely present in any well-specified codebook. How much of this detail a model actually needs depends on what it already knows from pre-training. Promise classification (P) is likely a *recognition-heavy* task: the concept is intuitive, and related notions of commitment and promise-keeping appear widely in natural language (Pan et al., 2023; Wang et al., 2024). Strategic thinking classification (L) is a *learning-heavy* task: level- k reasoning is a technical construct specific to behavioural game theory (Crawford et al., 2013), unlikely to be prevalent in pre-training corpora. We therefore ask:

RQ1. *For a given task, can existing codebooks serve as effective LLM prompts, and which of their components drive classification performance?*

Even if the codebook content provides sufficient detail, how it is presented (formatting, framing, phrasing) and whether the model reasons through the task may still

the sentiment of the following text. Use one of these labels: negative; positive.”). Our design space complements theirs by extending further into richer specification levels: our minimal prompt is already comparable to their well-specified condition, and our verbatim codebook prompts run to 1,000–1,500 words including theoretical frameworks, category definitions, and worked examples. Building on their finding that any task description beats no task description, we show that the further richness of a full researcher codebook continues to add value.

⁴See Appendix L for further discussion.

affect classification. The literature offers recommendations on these choices, typically without controlling for the level of informational detail present in the prompt. We test each intervention at both high and low levels of informational detail. This allows us to ask whether these “prompt engineering” techniques improve classification in their own right, or whether they only appear to help when the prompt lacks sufficient detail and the techniques partially fill that gap. This leads to our second question:

RQ2. *Do prompt engineering choices — formatting, framing, wording, and reasoning — improve classification beyond what the codebook content provides?*

The answers to both questions may depend on the model. A large model may extract value from a verbose codebook that overwhelms a smaller one; a smaller model may depend on prompt engineering choices that make no difference for a frontier model. We approximate model scale by total parameter count where it is disclosed; for the proprietary GPT models, where counts are undisclosed, we treat the ‘nano’ suffix as marking GPT 5.4-nano as smaller than GPT 5.4. We evaluate four models at two scale tiers within each family: GPT 5.4 and GPT 5.4-nano (proprietary), Qwen 3.5 397B and Qwen 3.5 9B (open-weight). This motivates our third question:

RQ3. *How does model scale affect classification performance and moderate the sensitivity to prompt content and design?*

4. Data

We use classification codebooks from three experimental economics studies, investigating two distinct classification tasks: identifying promises and inferring strategic reasoning levels. The *promise* task (P) classifies open-ended chat messages from a public goods game (Arad et al., 2024) as promise or empty talk; we use the 1,494 player-level utterances (drawn from 719 chat messages) on which both annotators agreed. The *level- k* tasks classify statements by strategic reasoning level: L_I classifies 493 messages from a voting game (Çelebi and Penczynski, 2023) into levels 0–3, and L_{II} classifies 851 messages from a set of coordination games (van Elten and Penczynski, 2020) into levels 0–5.

In all three tasks, two human annotators independently classified each item; ground truth is defined by the instances on which both annotators agreed. An item is scored as correct if the model’s prediction matches this agreed label, and a model’s accuracy is the proportion of items it classifies correctly.⁵

⁵For L_I and L_{II} , annotators were incentivised via a coordination game — paid based on agreement with

5. Design

All classification tasks use a standardized prompt template (Figure 1), informed by existing prompting guidelines and empirical findings on effective prompt design.⁶ The template begins with a general task description and a role persona, followed by experiment context, theory context, and classification instructions drawn from the codebook. Worked examples from the codebook are included in the n -shot conditions. The experiments that follow test each dimension of the template and their interactions, with the exception of the output format sections (constraints, classification coding, and output format), which are held constant across all conditions.⁷

In total, across all experiments, approximately 703,000 classification requests were submitted to the API (703,253 calls; 702,809 successful; 0.06% error rate), covering 1,182 unique model–task–condition combinations across 4 models. Experiments were conducted between 24 March and 6 April 2026.

5.1. Codebook-to-Prompt Pipeline

We convert each human classification codebook into a set of LLM prompts through a four-step pipeline applied uniformly to all four tasks. All LLM-assisted steps use Claude Opus 4.6.

Step 1: Filter. The raw codebook is processed with a structured filtering prompt (see Appendix J.1) that removes content relevant only to human annotators in a laboratory setting: greetings, payment details, software instructions, data-entry guidance, meta-instructions, and deadline warnings. Classification-relevant content — experiment mechanics, category definitions, and worked examples — is retained verbatim. As a light reframing, second-person laboratory instructions (“Your task is to...”) are converted to third-person descriptions (“Each participant...”). No information is added or paraphrased beyond this reframing.

Step 2: Separate. The filtered codebook is manually divided into three named sections: *experiment context* (game mechanics, payoffs, communication protocol), *theory context* (the level- k framework, L-tasks only), and *classification instructions* (category

each other — and informed of their disagreements between two rounds of classification. Houser and Xiao (2011) show that this incentive structure makes annotators more responsive to fine-grained changes in classification instructions. The P annotations were produced without such incentivisation.

⁶Each template component is motivated by specific findings in the prompting literature. See Appendix K for the full rationale and citations.

⁷Constraints and output format enforce structured JSON responses, which has been shown to improve classification accuracy (Tam et al., 2024) and lets the classification label be read directly from the response without any string parsing.

```

# General Task
- Classify <X> in <E>

# Role Persona
- Act as a behavioural economist specialized in text
classification, concept <C> and game <G>

# Context
## Experiment
- <Game mechanics>
- <Experimental design / Decision process>
- ...
## Theory
- <Theoretical framework>
- ...

# Classification Task
- Classify <X> as <Y> given criteria: <Y1, Y2, ...>
- ...

# Classification Coding
- Code <X> as <Z>
- ...

# Examples
- <Example text> <classification>
- ...

# Constraints
- Provide your response using the JSON format specified below
- Do not provide explanations

# Output Format
{'classification': '<value>'}

```

Note: Theory subsection is included only for *L*-tasks. Examples are included only in *n*-shot conditions. All prompts use markdown formatting with itemized lists.

Figure 1: Prompt template.

definitions and assignment rules). Worked examples are extracted into a separate file for use in few-shot conditions. This separation allows each section to be independently varied in subsequent experiments.

Step 3: Compress. Each section is iteratively compressed to two intermediate levels (C1 at approximately 50–60% and C2 at approximately 25–30% of the verbatim word count) using an LLM with a detailed compression prompt (see Appendix J.2). The prompt is applied twice in sequence (verbatim → C1, C1 → C2) and instructs the model to only remove content — never to add, infer, or paraphrase — while preserving a task-specific list of key technical terms at every level. Table 2 shows that embedding cosine similarity to the verbatim text remains above 0.80 at all compression levels, and that all key terms are preserved across all tasks and all compression steps.

Step 4: Assemble. At runtime, prompt variants are assembled from their component sections. The assembler concatenates sections in a fixed order (framing header, experiment context, theory context, classification instructions, examples) and applies the requested content level and format treatment, producing a single system prompt. The text to classify is provided in the user prompt.⁸

5.2. Models

We evaluate four models: two proprietary and two open-weight (Table 1). The GPT 5.4 pair (5.4 and 5.4-nano) represents a frontier closed-source family at two capability tiers; parameter counts are not disclosed. The Qwen 3.5 pair (397B and 9B) represents a state-of-the-art open-weight family at two known scales — 17B active parameters (MoE) and 9B (dense) — enabling a within-family scale comparison. GPT models are accessed through Azure OpenAI Service; Qwen models through Together AI and OpenRouter.

Model	Company	Total	Active	Architecture	Released	Provider
GPT 5.4	OpenAI	—	—	closed	Mar 2026	Azure OpenAI
GPT 5.4-nano	OpenAI	—	—	closed	Mar 2026	Azure OpenAI
Qwen 3.5 397B	Alibaba	397B	17B	MoE	Feb 2026	Together AI / OpenRouter
Qwen 3.5 9B	Alibaba	9B	9B	dense	Feb 2026	Together AI / OpenRouter

Note: Active params = parameters active per forward pass (equal to total for dense models). GPT parameter counts are not disclosed. GPT models support reasoning at multiple effort levels; Qwen models support a binary reasoning toggle.

Table 1: Models evaluated in this study.

5.3. Experiments

We conduct six experiments, each targeting a different dimension of prompt design. All experiments use the same items and models, enabling paired comparisons throughout. We represent each prompt configuration as a tuple (c, x, t, e) , where c is the classification instruction level, x is the experiment context level, t is the theory context level (L -tasks only), and e is the example condition. Subscripts denote detail: verbatim (V), compressed (C_1, C_2), or absent (N) for context and theory; verbatim (V) or simple (S) for classification instructions; zero-shot ($\emptyset$) or n -shot (n) for examples. Two anchoring configurations are used throughout:

⁸See Appendix H for details.

- **Verbatim** ($c_V, x_V, t_V, \emptyset$): the full codebook content — classification instructions, context, and theory — transcribed verbatim, without examples.
- **Minimal** ($c_S, x_N, t_N, \emptyset$): one-line category descriptions only, with no context, theory, or examples.

5.3.1. Content Ablation

Each codebook is decomposed into three independently varied information sources: classification instructions (verbatim or simple), experiment context (V/C1/C2/N), and theory context (L -tasks only; V/C1/C2/N). Crossing these factors yields 8 variants per P -task (4×2 , no theory) and 32 per L -task ($4 \times 4 \times 2$).

Classification instructions are tested at two levels. *Verbatim* (V) transcribes the codebook’s category definitions in full — typically 6–15 bullet points per category covering inclusion criteria, edge cases, and boundary rules. *Simple* (S) reduces codebook’s category definitions to a single definitional sentence (e.g., “Classify the player’s level of strategic thinking as 0, 1, 2, or 3”), retaining only 3–11% of the verbatim word count.

Source	Level	Cosine	Words	Key terms
Exp. context	V	1.000	293 (100%)	10
	C1	0.857	174 (59%)	10/10
	C2	0.831	97 (33%)	10/10
	N	—	0 (0%)	—
Theory	V	1.000	735 (100%)	9
	C1	0.901	374 (51%)	9/9
	C2	0.804	158 (21%)	9/9
	N	—	0 (0%)	—
Cls. instructions	V	1.000	—	—
	S	0.724	— (3%)	—

Note: Cosine similarity computed against the verbatim original using the `text-embedding-3-large` model of OpenAI. Key terms = task-critical tokens (game labels, payoff numbers, category names) enforced by the compression prompt. Word counts shown for L_I ; other tasks in Appendix A.3.

Table 2: Semantic retention by prompt level (L_I).

Experiment context and theory context are each compressed to two intermediate levels (C1, C2) using the procedure described in Section 5.1. Table 2 shows that em-

bedding cosine similarity to the verbatim text remains above 0.80 at all compression levels and all key terms are retained.⁹

5.3.2. Formatting and Framing

Format. Five treatments are applied to identical textual content: markdown with headers and bullet lists (baseline), titles with bullet lists, titles with prose paragraphs, flat bullet lists without headers, and plain unformatted text. Each format is tested at both verbatim and minimal content levels, yielding $5 \text{ formats} \times 3 \text{ tasks} \times 2 \text{ content levels} = 30$ conditions per model (see Appendix F.2 for details).

Framing. Two framing elements are tested: a general task description (GT) that names the experimental domain (e.g., “classify messages from a coordination game”) and a role persona (RP) that assigns a domain identity (e.g., “behavioral economist with expertise in level- k reasoning”). Three removal conditions (GT only, RP only, Neither) are compared against the full-framing baseline at both content levels, yielding $4 \times 3 \times 2 = 24$ conditions per model (see Appendix F.3 for details).

5.3.3. Examples

Each codebook’s worked examples are added to the prompt in an n -shot format. The number of examples is determined by what each codebook provides: 21 for P (10.5 per label), 13 for L_I (3.25 per level), and 5 for L_{II} (1.25 per level). Examples are tested at both content levels (minimal and verbatim), enabling a direct comparison of content versus examples as alternative sources of classification information, yielding $2 \text{ example conditions} \times 2 \text{ content levels} \times 3 \text{ tasks} = 12$ conditions per model.

5.3.4. Reasoning

Model reasoning is enabled at multiple effort levels. GPT models support five levels (off, low, medium, high, extra-high); Qwen models support a binary toggle (on/off). Reasoning is crossed with content level (minimal/verbatim) and example provision (0-shot/ n -shot), yielding $5 \times 2 \times 2 \times 3 = 60$ reasoning variants per GPT model and $2 \times 2 \times 2 \times 3 = 24$ per Qwen model (baselines shared with other experiments) (see Appendix F.4 for details).

⁹Full cosine similarities for all tasks are in Appendix A.3; the full factorial design is in Appendix F.1.

5.4. Metrics

LLMs are not fully deterministic: even with the same prompt, the same input text, and temperature set to 0, repeated runs can produce different classifications. This arises from numerical non-associativity in floating-point operations during inference, which makes token selection sensitive to hardware-level variation when the gap between the top candidates is small (Yuan et al., 2025). In practice, studies running the same prompt on the same items 30–50 times report that 8–12% of classifications change across runs, with more complex tasks showing even higher variability (Atil et al., 2024; Wang and Wang, 2025). A few-point accuracy gap between two prompts can therefore reflect run-to-run noise rather than a genuine treatment effect. We address this by using McNemar’s test throughout: each comparison pairs the *same items* under two conditions, and the test asks whether the items on which the two conditions disagree split asymmetrically, with one condition systematically more often correct than the other. This paired design controls for item difficulty and isolates the treatment effect from run-to-run variation. Significance is assessed at $p < 0.001$ throughout. Accuracy is computed per item as described in Section 4; all accuracy estimates are reported with 95% Wilson confidence intervals (Wilson, 1927). Additionally, instead of accuracy, Appendix M reports tables and decompositions in terms of the support-weighted macro F1 score with paired bootstrap confidence intervals; the qualitative findings are unchanged, and the F1 replication is discussed there alongside the accuracy comparison.

6. Results

6.1. Content

	Minimal			Verbatim		
	P	L_I	L_{II}	P	L_I	L_{II}
GPT 5.4	73.2 [70.9, 75.3]	47.4 [43.0, 51.8]	49.6 [46.2, 52.9]	74.2 [71.9, 76.3]	81.3 [77.7, 84.5]	81.0 [78.2, 83.5]
GPT 5.4-nano	60.9 [58.4, 63.4]	52.5 [48.1, 56.9]	53.1 [49.8, 56.5]	64.3 [61.8, 66.6]	70.2 [66.0, 74.1]	68.9 [65.7, 71.9]
Qwen 397B	70.5 [68.1, 72.7]	50.1 [45.7, 54.5]	43.6 [40.3, 46.9]	66.2 [63.8, 68.6]	83.0 [79.4, 86.0]	74.6 [71.6, 77.4]
Qwen 9B	64.1 [61.7, 66.5]	53.1 [48.7, 57.5]	38.8 [35.6, 42.1]	63.3 [60.8, 65.7]	70.8 [66.6, 74.6]	64.4 [61.1, 67.5]

Note: 95% Wilson CIs below each estimate. **Bold** = highest value per task within each content level; tied values (McNemar $p \geq 0.001$) share bold.

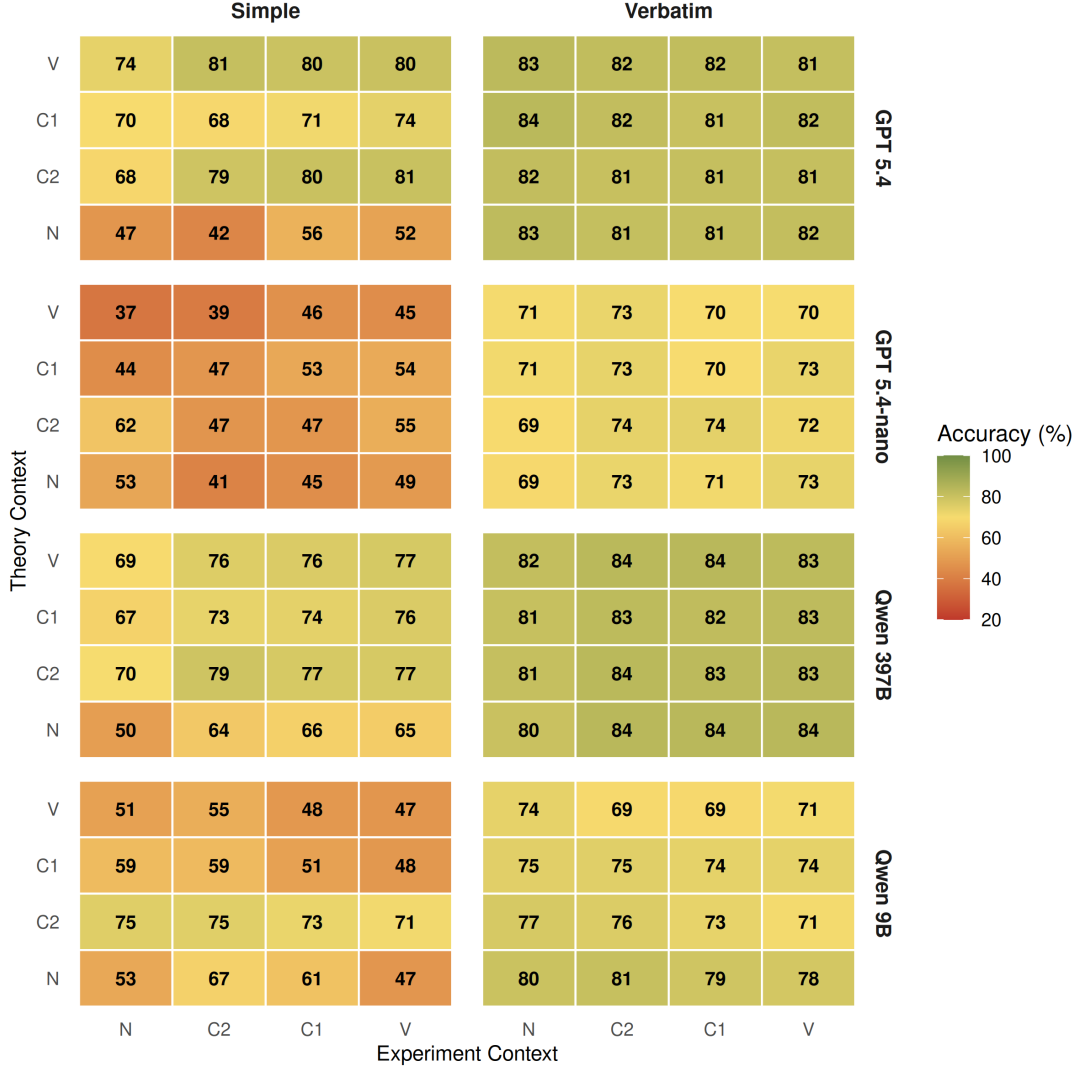
Table 3: Accuracy (%) at the minimal and verbatim prompts.

Table 3 reports accuracy under the minimal and the verbatim prompt. On P , the verbatim prompt yields no significant improvement over minimal: GPT 5.4 moves from 73.2% to 74.2%, and the other models show similarly small or even negative changes. Promise classification thus appears to be recognition-heavy: the model can often map the label to the relevant concept without needing further information.¹⁰ The L -tasks, by contrast, are fundamentally learning-heavy. Under the minimal prompt, performance is poor but improves as the information needed to interpret the categories is provided.

Figure 2 shows how theory and experiment context interact on L_I across different levels of classification detail (L_{II} surfaces in Appendix A.1). Under simple classification instructions in the left column, theory contributes substantially on its own, whereas experiment context alone either hurts performance or marginally benefits it. Once theory is present, however, experiment context becomes effective, indicating that the two components complement each other: theory helps define the categories, while experiment context helps map them onto the task. Together, they can partially substitute for missing classification instructions. By contrast, under verbatim classification instructions in the right column, the surface is largely flat for GPT 5.4 and Qwen 397B, with only small positive differences across context conditions. This suggests that theory

¹⁰A variance decomposition (see Appendix A.2) confirms this: on P , model choice accounts for nearly all explained variance (98%), while on the L -tasks classification instructions dominate (58% on L_I).

and experiment context matter primarily when classification instructions are under-specified.



Note: Accuracy (%) by experiment context (x-axis) and theory context (y-axis), separately under simple (left) and verbatim (right) classification instructions. Context levels: N = none, C2 $\approx$ 25%, C1 $\approx$ 50%, V = verbatim.

Figure 2: L_1 information surface.

Model scale further moderates this pattern. GPT 5.4 and Qwen 397B benefit from additional content when classification instructions are absent and still benefit marginally when classification instructions are present. Smaller models, by contrast, perform best when classification instructions are present but context is limited or absent, reaching their highest accuracy at the lowest (C2) compression level (Figure 2). This indicates that they are less able to make effective use of the full contextual information (see

Appendix A.1 for L_{II} information surface).

Result 1. *Experiment and theory contexts partially complement each other and partially substitute for classification instructions. Larger models benefit from richer content, whereas smaller models perform better with less.*

6.2. Formatting and Framing

	Formatting				Framing			
	Minimal		Verbatim		Minimal		Verbatim	
	+	−	+	−	+	−	+	−
GPT 5.4	.17	.08	.00	.08	.33	.11	.33	.00
GPT 5.4-nano	.08	.00	.08	.00	.33	.22	.00	.00
Qwen 397B	.08	.08	.00	.00	.11	.44	.00	.00
Qwen 9B	.33	.17	.00	.00	.44	.00	.00	.11

Note: Proportion of all comparisons that are significantly positive (+) or negative (−) at $p < 0.001$. Formatting: 4 treatments vs. markdown baseline (12 per cell: 4 formats $\times$ 3 tasks). Framing: 3 conditions (both, GT only, RP only) vs. no-framing baseline (9 per cell: 3 conditions $\times$ 3 tasks). Mean effects and dispersion in Appendix B.

Table 4: Formatting and framing: proportion of significant effects.

Formatting. On verbatim prompts, switching from markdown to any of the four alternative formats produces almost no significant effects for any model (Table 4, left half). On minimal prompts, Qwen 9B shows scattered significance (.33 positive, .17 negative) but with no consistent direction; the other three models remain largely unaffected. *Framing.* On verbatim prompts, framing effects are again negligible (Table 4, right half). On minimal prompts, the pattern is model-dependent: Qwen 9B shows .44 significant positive effects (driven by domain cues in the general task description that are absent from the sparse prompt), while Qwen 397B shows .44 significant negative. For GPT 5.4, the few significant framing effects are driven entirely by the general task description (GT) rather than the role persona.¹¹

¹¹On verbatim prompts, GPT 5.4 shows three significant positive effects, all GT-driven: GT only on P (+2.9 pp), GT only on L_{II} (+3.6 pp), and Both on P (+2.8 pp). The GT sentence names the domain (“public goods game”, “coordination game”), providing a keyword anchor even when the full codebook is present. Role persona alone never reaches significance on any model on verbatim prompts. See Appendix B for details.

Result 2. *Neither formatting nor framing reliably affects classification accuracy. On verbatim prompts, effects are negligible. On minimal prompts, significance increases but the direction is inconsistent across models and tasks.*

6.3. Examples

	P	L_I	L_{II}
Δ_{Ex}	+16.6*	+37.4*	+20.4*
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+15.6*	+3.4	-10.9*
$\Delta_{\text{Ex} \text{Content}}$	+14.0*	+2.6	+0.7
$\Delta_{\text{Content} \text{Ex}}$	-1.6	-0.8	+11.6*

Note: * = McNemar $p < 0.001$. All values in pp. Δ_{Ex} : gain from adding examples to the minimal prompt. $\Delta_{\text{Ex}|\text{Content}}$: gain from adding examples to the verbatim prompt. $\Delta_{\text{Ex}} - \Delta_{\text{Content}}$: minimal+examples minus verbatim+zero-shot (positive = examples alone outperform content alone). $\Delta_{\text{Content}|\text{Ex}}$: verbatim+examples minus minimal+examples (positive = content adds value on top of examples).

Table 5: GPT 5.4: effect of examples.

Table 5 decomposes the contributions of content and examples¹² for GPT 5.4. Examples alone produce large accuracy gains on every task ($\Delta_{\text{Ex}} = +16.6$ pp on P , $+37.4$ pp on L_I , $+20.4$ pp on L_{II}), comparable to or larger than the gain from adding verbatim content alone. Content and examples convey overlapping information about what the categories mean, making them partial substitutes. Once verbatim content is already in the prompt, the value of additional examples scales with the number of examples per label the codebook supplies¹³: $\Delta_{\text{Ex}|\text{Content}}$ is $+14.0$ pp on P , $+2.6$ pp on L_I , and a non-significant $+0.7$ pp on L_{II} . The same monotonic pattern appears under weighted F1 and across all four models.¹⁴

Result 3. *Examples alone produce large accuracy gains across all tasks. Content and examples are partial substitutes. The accuracy gain from examples given content scales with the example count.*

¹²We also varied the in-prompt format of the examples (markdown headers, XML tags, plain delimiters, arrow notation, multi-turn conversation); design and full results in Appendix C.4.

¹³ P provides 21 worked examples (10.5 per label), L_I 13 (3.25 per level), and L_{II} 5 (1.25 per level); see Appendix C.1. A task-intrinsic effect — P being more pattern-recognition and L more theory-laden — may also contribute to this monotonic decline; our design does not let us isolate it from the example-count effect.

¹⁴See Appendix M for details.

	P	L_I	L_{II}
GPT 5.4	88.2 [86.4, 89.7]	84.0 [80.5, 86.9]	81.7 [78.9, 84.1]
GPT 5.4-nano	72.0 [69.6, 74.2]	78.3 [74.4, 81.7]	70.7 [67.6, 73.7]
Qwen 397B	86.7 [84.9, 88.4]	83.6 [80.0, 86.6]	77.8 [74.9, 80.5]
Qwen 9B	78.5 [76.4, 80.5]	82.8 [79.2, 85.8]	68.2 [64.9, 71.2]

Note: Verbatim content + n -shot examples, no reasoning. 95% Wilson CIs below each estimate.

Bold = highest value per task; tied values (McNemar $p \geq 0.001$) share bold.

Table 6: Accuracy (%) with the full codebook: verbatim content and worked examples.

Table 6 represents what a researcher obtains by faithfully transcribing the codebook with its worked examples (with the minor LLM-assisted filtering described in Section 5.1). On P and L_I , the open-weight Qwen 397B matches the proprietary GPT 5.4 (McNemar $p \geq 0.001$), and even Qwen 9B reaches 83% on L_I . The tasks separate clearly by difficulty: all models achieve 72–88% on P and 78–84% on L_I , but L_{II} drops to 68–82%, with the gap between frontier and small models widening from ≈ 2 pp on L_I to ≈ 13 pp on L_{II} .

6.4. Reasoning

Can reasoning compensate for missing codebook content — that is, can a model that “thinks harder” overcome an underspecified prompt? Table 7 decomposes reasoning and content effects in the same way Table 5 decomposed examples and content. The answer is no.

On L -tasks, content alone clearly outperforms reasoning alone ($\Delta_{\text{Rea}} - \Delta_{\text{Content}}$: -45.4 pp on L_I , -11.6 pp on L_{II}). On L_I , reasoning on the minimal prompt actually *hurts*: $\Delta_{\text{Rea}} = -11.5$ pp, meaning that the model’s attempt to reason without adequate classification instructions produces worse results than no reasoning at all.

The conditional effects are asymmetric: content remains highly valuable when reasoning is active ($\Delta_{\text{Content}|\text{Rea}}$: $+46.7$ pp on L_I , $+11.6$ pp on L_{II}), but reasoning adds nothing when content is already present ($\Delta_{\text{Rea}|\text{Content}}$: $+1.2$ pp n.s. on L_I , exactly 0.0 pp on L_{II}). The same pattern holds across all four models (see Appendix D.3): $\Delta_{\text{Rea}|\text{Content}}$ is near-zero on L -tasks for GPT 5.4, GPT 5.4-nano, and Qwen 397B, with only Qwen 9B showing a significant effect on L_I ($+10.5$ pp). On P , smaller models show significant reasoning-given-content gains (GPT 5.4-nano: $+6.5$ pp, Qwen 397B: $+8.4$ pp,

Qwen 9B: +9.4 pp), suggesting that reasoning can partially compensate for weaker pre-training representations of familiar concepts, but not for missing domain-specific theory.

	P	L_I	L_{II}
Δ_{Rea}	+9.5*	-11.5*	+19.7*
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+8.5*	-45.4*	-11.6*
$\Delta_{\text{Content} \text{Rea}}$	-9.7*	+46.7*	+11.6*
$\Delta_{\text{Rea} \text{Content}}$	-1.2	+1.2	+0.0

Note: * = McNemar $p < 0.001$. All values in pp, 0-shot only, no examples. For GPT models we test low, medium, high, and extra-high reasoning effort; we report high to match Qwen’s reasoning token output (≈ 750 tokens). No incremental step from low to extra-high produces a significant improvement (see Appendix D.1). Δ_{Rea} : minimal+reasoning minus minimal 0-shot. $\Delta_{\text{Rea}} - \Delta_{\text{Content}}$: minimal+reasoning minus verbatim 0-shot (negative = content alone outperforms reasoning alone). $\Delta_{\text{Content}|\text{Rea}}$: verbatim+reasoning minus minimal+reasoning. $\Delta_{\text{Rea}|\text{Content}}$: verbatim+reasoning minus verbatim 0-shot. Remaining models in Appendix D.3.

Table 7: GPT 5.4: content vs. reasoning substitutability.

With reasoning enabled on verbatim prompts at both 0-shot and n -shot, GPT 5.4 shows no significant effect on any task at any condition.¹⁵ Smaller models benefit on P at both shot conditions, but on L -tasks their gains appear only at 0-shot and disappear once examples are present: examples render the reasoning gains redundant.¹⁶

Result 4. *Reasoning cannot substitute for codebook content and does not improve frontier-model classification. On well-specified prompts, smaller models gain from reasoning only on the P -task; their L -task gains at zero-shot disappear once examples are present.*

7. Discussion

7.1. Summary of Findings

Faithful transcription of a codebook — including category definitions, context, theory, and the worked examples it supplies — produces 82–88% agreement with the human-agreed ground truth across our tasks. What drives this accuracy is the level of

¹⁵see Appendix D for additional results

¹⁶Adding examples costs $\approx 0.3\text{--}0.4 \times$ the prompt (input-rate tokens), comparable to low reasoning, yet produces accuracy gains an order of magnitude larger on L -tasks. See Appendix I for the full cost comparison.

informational detail in the prompt. Detailed classification instructions are the dominant component, while experiment and theory contexts partially complement each other and together can partially substitute for the classification instructions. Examples on their own also produce large gains. The information they carry partially overlaps with that of the codebook content, making the two partial substitutes: once a verbatim codebook is in place, the value of additional examples scales with how many examples per label the codebook supplies.

How effectively a model utilizes this informational detail depends on both the task and its own scale. On learning-heavy tasks, detailed instructions are essential; on recognition-heavy tasks, model choice explains more of the variation in performance than prompt content. Larger models tolerate surface variation better and extract more value from detailed content. Smaller models are more sensitive, and in some cases, additional detail can even degrade their performance.

How the information is presented is less of a concern for capable models. Smaller models exhibit prompt brittleness, however the direction of their sensitivity to surface variations is inconsistent across tasks and treatments. Similarly, introducing model reasoning does at best partially compensate for missing content, and adds little to no benefit for larger models once detailed content is present.

These results yield an actionable recommendation for applied researchers: shift the focus from prompt engineering to codebook quality. The most reliable classification strategy is to pair a frontier model with a well-defined codebook that provides the informational detail a human annotator would need.

7.2. Beyond the Codebook

A researcher adapting a human codebook for an LLM might feel that the original is too sparse in places and want to elaborate, adding the kind of explanation, rationale, or worked-out criteria that a careful human reader would also benefit from. We systematically expanded the codebook in this way¹⁷ and saw no significant change for GPT 5.4; injecting irrelevant content of matched length had no effect either, suggesting that frontier models filter out what is not useful. Smaller models respond to both treatments at 0-shot, but the effects largely vanish once examples are present. Similarly, paraphrasing produces effects below 1 pp for all models at 0-shot; at n -shot, smaller

¹⁷We tested three perturbations of the prompt text itself: (i) verbose elaboration (+50–275% words of relevant detail) plus a matched noise control of irrelevant content at the same length; (ii) round-trip back-translation (English → Turkish/German → English); (iii) the raw unprocessed codebook used directly as the prompt. Designs in Appendix F.5; significance tables in Appendix E (verbose/noise) and Appendix E.2 (paraphrase).

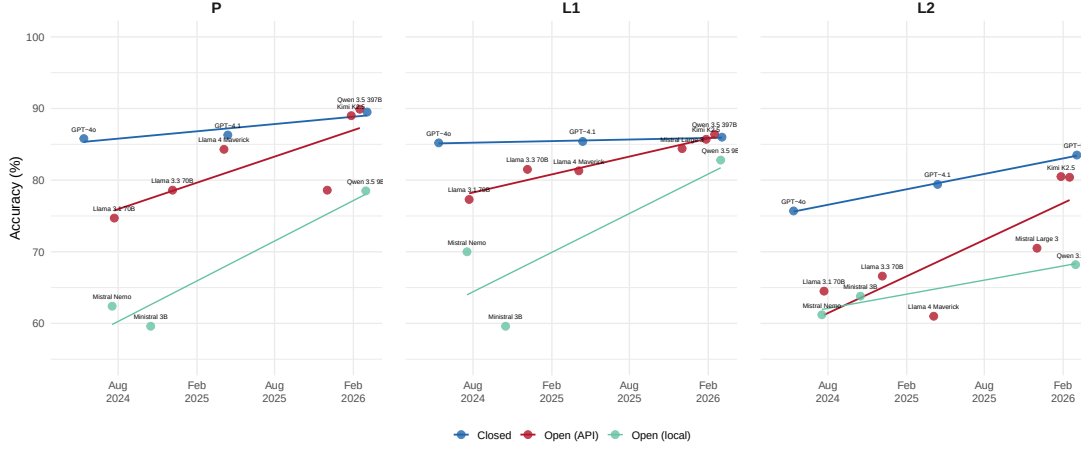
models show occasional sensitivity, again without consistent direction. And the raw unprocessed codebook, including payment instructions and software notes intended for human annotators, performs comparably to the cleaned version for GPT 5.4.

As Sections 1 and 2 discuss, the apparently conflicting reports of prompt brittleness in the literature largely correspond to underspecified prompts or weaker models. Our results extend the point beyond formatting: verbose elaboration, irrelevant noise, paraphrasing, and uncleaned procedural content all fail to move the needle once the codebook is detailed and the model is capable.

A parallel observation comes from compound AI systems: Zhang et al. (2026) find that prompt optimisation is already a coin flip on tasks where the model’s default behaviour is well-aligned, and argue that instruction tuning and reinforcement learning from human feedback increasingly compress diverse input phrasings into consistent outputs, shrinking the headroom that surface-level optimisation can exploit. We see the same shrinking headroom in the classification setting: much of the prompt-engineering literature was written for earlier, less capable models, and the domain in which it still moves the needle has been narrowing as models improve.

This robustness reflects the quality of our codebooks: experimental-economics codebooks define categories through formal models, give inclusion and exclusion criteria, and provide worked examples grounded in the same theory. Codebooks elsewhere in the social sciences are often sparser, with many labels, short definitions, overlapping categories, and no formal theoretical grounding. In those settings the codebook may genuinely be underspecified, and elaboration or restructuring may be warranted. Our content-ablation results (Section 6.1) make the asymmetry concrete: removing detail from an already-detailed codebook drops accuracy sharply on theory-dependent tasks, so the model is not indifferent to content — only to surplus content. For researchers in other domains, the practical question is therefore not whether to invest time in prompt engineering, but whether to invest it in enriching the codebook itself — adding the category definitions, boundary cases, and worked examples that our codebooks already provide.

This raises a diagnostic question: how can a researcher tell whether their prompt is already well-specified or still underspecified? Our results suggest that robustness to surface variation is itself the signal: a well-specified prompt produces consistent classifications regardless of how it is formatted, framed, or paraphrased, while an underspecified prompt does not. Paraphrasing is one such surface variation, and it is exactly how Barrie et al. (2025) measure prompt stability. What their framework does not address is why one prompt is stable and another is not. Our results point to a candi-



Note: Accuracy on verbatim n -shot prompts (with reasoning where available) across model generations. Blue = proprietary GPT family; red = frontier open-weight models; green = small open-weight models ($\leq 10B$) that can run on consumer hardware.

Figure 3: Model performance over time by task.

date explanation: stability emerges once the prompt is saturated with the information the model can effectively use, after which further content or rewording yields only marginal gains. Practical tests for when a given prompt and model have reached this saturation point are a natural direction for future work.

7.3. Open or Closed Models

Figure 3 plots accuracy over time for models featured in the prompting literature we cite, alongside additional models we evaluated to broaden the comparison.¹⁸ Three trends are visible. First, frontier open-weight models (red) have converged toward proprietary performance (blue) over two years, with the steepest gains on L -tasks: Qwen 3.5 397B now matches GPT 5.4 on P and L_I . Second, small open-weight models that can run on consumer hardware (green, $\leq 10B$ parameters) are also improving rapidly — Qwen 3.5 9B reaches 82–84% on L_I , a level that required frontier models a generation earlier. Third, GPT 5.4 retains a clear lead on the hardest task (L_{II}), where the gap between proprietary and open models is still substantial.

A closing accuracy gap does not mean open models are equally dependable in practice. Smaller open-weight models remain more sensitive to surface variations: formatting, framing, and wording changes that have no effect on GPT 5.4 can produce

¹⁸Beyond the four models analysed in this paper, we ran the same verbatim n -shot classification on GPT 4o, GPT 4.1, Llama 3.1 70B, Llama 3.3 70B, Llama 4 Maverick, Kimi K2.5, Mistral Large 3, Mistral NeMo (12B), and Minstral 3B. These additional runs are used here for the timeline comparison only; the factorial experiments reported in the results are limited to the four featured models.

significant shifts for Qwen 9B (Section 6.2). This matters for the distinction between *reproducibility* (repeating the same analysis with the same prompt and model) and *replicability* (arriving at the same conclusions with reasonable variations in prompt design). An open model sensitive to surface variations may reproduce well (the same prompt and model weights yield the same outputs) yet fail to generalise when another researcher adapts the prompt to a new setting with minor formatting, framing, or phrasing changes.¹⁹ GPT 5.4’s insensitivity to these choices makes it more replicable, not just more accurate.

The main argument against proprietary models is reproducibility: if the provider deprecates the model, the exact same classification can no longer be reproduced. But how bad is this problem in practice? Inter-model agreement across three consecutive GPT generations (4o, 4.1, 5.4) on identical items is high.²⁰ A researcher who classified data with GPT 4o and later re-ran the analysis with GPT 5.4 would obtain higher accuracy, potentially greater robustness to prompt variation, and classification agreement comparable to substituting one trained human annotator for another.²¹ This is not perfect reproducibility, but it is a predictable and improving trajectory — and no worse than the inter-coder reliability one would accept when replacing one human annotator with another (Li et al., 2025).

Even our largest open-weight model, Qwen 3.5 397B, remains sensitive to surface-level prompt variation in a way that GPT 5.4 is not (Section 6.2). Reaching frontier-level robustness therefore appears to require a further scale jump for open models, plausibly to the trillion-parameter range that closed frontier models such as GPT 5.4 are believed to inhabit. On the open side, Moonshot’s Kimi K2.5 currently exemplifies that range.²²

¹⁹No LLM classification is perfectly reproducible: even with identical prompts, temperature 0, and the same model, run-to-run variation produces flip rates of 2–12% depending on the model and task (Atil et al., 2024; Wang and Wang, 2025). We use “reproduce” here in the weaker sense of obtaining substantively similar results.

²⁰We measure inter-model agreement using Krippendorff’s α , a reliability coefficient that accounts for chance agreement; $\alpha = 1$ indicates perfect agreement, and values above $\alpha \approx 0.7$ are commonly read as substantial agreement (see e.g. Barrie et al., 2025). On our data, adjacent GPT generations reach $\alpha = 0.89$ – 0.91 on P and 0.84 – 0.88 on L_I ; the full table is in Appendix G.

²¹For reference, on the P task the two human annotators in our source data (Arad et al., 2024) reach Krippendorff’s $\alpha = 0.48$ before disagreements were resolved — considerably below the cross-generation GPT agreement of 0.89 – 0.91 reported above.

²²Kimi K2.5 was the only other state-of-the-art open-weight model released as recently as the Qwen 3.5 series at the time of writing. We did not include it in our factorial experiments because its non-reasoning mode is forced to temperature 0.6, and we did not want a model whose outputs come from a temperature level other than 0 to be mixed with the rest of our temperature-0 runs.

7.4. Limitations

Experimental-economics text is a best-case setting for codebook-based classification, and our results should be read with this in mind. Three features of our setting are unlikely to hold jointly in other social-science domains. Beyond the codebook quality discussed in Section 7.2, the text we classify is tightly constrained: subjects communicate within fixed experimental rules with known payoffs and structured protocols, producing a narrow vocabulary and a clear mapping between language and the concepts being classified. Tweets, parliamentary speeches, Reddit threads, or interview transcripts are far more variable in style, length, and ambiguity, and a model that ignores surface variations in our setting may not do so once the text itself becomes more variable. We are also able to provide the full experimental and theoretical context in the prompt because we designed the experiments; most applied settings lack this, leaving the researcher with a codebook and the text but no complete description of the data-generating process.

These conditions make our null finding on surface features a strong result within this setting; whether the same holds in domains where any of these conditions is relaxed remains an open question. The content-ablation results show what happens when prompt detail drops: accuracy on L -tasks falls from above 80% to below 50% when classification instructions are reduced to one line. Researchers working with codebooks closer to our minimal condition than our verbatim condition, as many social-science codebooks are, may find that the prompt design choices we show to be irrelevant here do matter for them. This can stem from two sources: their codebook may not yet give the model enough information to ignore how it is presented, or the concepts being classified may be intrinsically more ambiguous, limiting what any codebook can fully specify.

8. Conclusion

For frontier models on codebook-based classification tasks, the practical recommendation is clear: transcribe the codebook faithfully, include the worked examples it provides, and use a capable model. This yields 82–88% agreement with humans across the four models we test. Detailed classification instructions and worked examples are the two drivers of accuracy. The effect of surface variations on classification outcome becomes negligible as the level of informational detail of the prompt increases. Furthermore, model reasoning does not substitute for missing detail, adding little for capable models once detail is present.

These findings suggest a shift in attention from prompt engineering toward two factors that actually matter: the level of informational detail of the classification information provided to the model, and the choice of a model capable of processing that information reliably. The effort conventionally allocated to formatting, persona assignment, and iterative wording refinement is better invested in ensuring that the classification instructions are clear and detailed.

References

- Arad, Ayala, David Hugh-Jones, and Stefan P. Penczynski**, “Communication is informative about cooperation,” Technical Report, University of East Anglia 2024. Working paper.
- Atil, Berk, Aleya Aykent, Breck Chittams, Jesse Baldwin, and Aylin Caliskan**, “Non-Determinism of “Deterministic” LLM Settings,” *arXiv preprint arXiv:2408.04667*, 2024.
- Atreja, Shubham, Joshua Ashkinaze, Lingyao Li, Julia Mendelsohn, and Libby Hemphill**, “Prompt Design Matters for Computational Social Science Tasks but in Unpredictable Ways,” in “Proceedings of the International AAAI Conference on Web and Social Media,” Vol. 19 2025, pp. 122–145.
- Barrie, Christopher, Evangelia Palaiologou, and Petter Törnberg**, “Prompt Stability Scoring for Text Annotation with Large Language Models,” *arXiv preprint arXiv:2407.02039*, 2025.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell et al.**, “Language models are few-shot learners,” *Advances in neural information processing systems*, 2020, 33, 1877–1901.
- Camerer, Colin F., Teck-Hua Ho, and Juin-Kuan Chong**, “A Cognitive Hierarchy Model of Games,” *The Quarterly Journal of Economics*, August 2004, 119 (3), 861–898.
- Carlson, Kevin D and Vanessa C Burbano**, “The Use of LLMs to Annotate Data in Management Research,” *Strategic Management Journal*, 2026, 47 (3), 699–725.
- Çelebi, Can and Stefan P. Penczynski**, “Strategic Thinking in Jury Decisions: An Experimental Study,” Technical Report, School of Economics, University of East Anglia, Norwich, UK. 2023.
- Chae, Youngjin and Thomas Davidson**, “Large language models for text classification: From zero-shot learning to fine-tuning,” *Sociological Methods & Research*, 2025.
- Charness, Gary and Martin Dufwenberg**, “Promises and partnership,” *Econometrica*, 2006, 74 (6), 1579–1601.
- Clavié, Benjamin, Alexandru Ciceu, Frederick Naylor, Guillaume Soulié, and Thomas Brightwell**, “Large language models in the workplace: A case study on prompt engineering for job type classification,” in “International Conference on Applications of Natural Language to Information Systems” Springer 2023,

pp. 3–17.

Crawford, Vincent P., Miguel A. Costa-Gomes, and Nagore Iriberri,

“Structural Models of Nonequilibrium Strategic Thinking: Theory, Evidence, and Applications,” *Journal of Economic Literature*, September 2013, 51 (1), 5–62.

Dunivin, Zackary, “Scaling Hermeneutics: Qualitative Coding with LLMs,” *EPJ Data Science*, 2025.

Efrat, Avia and Omer Levy, “The turking test: Can language models understand instructions?,” *arXiv preprint arXiv:2010.11982*, 2020.

Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli, “ChatGPT outperforms crowd workers for text-annotation tasks,” *Proceedings of the National Academy of Sciences*, 2023, 120 (30), e2305016120.

Halterman, Andrew and Katherine A Keith, “Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts,” *Political Analysis*, 2025.

Han, Yuchen, Yucheng Wu, and Jeffrey Willard, “Effect of Selection Format on LLM Performance,” *arXiv preprint arXiv:2503.06926*, 2025.

He, Jia, Mukund Rungta, David Koleczek, Arshan Sekhon, and Siddhant S Hasan, “Does Prompt Formatting Have Any Impact on LLM Performance?,” *arXiv preprint arXiv:2411.10541*, 2024.

Houser, Daniel and Erte Xiao, “Classification of natural language messages using a coordination game,” *Experimental Economics*, 2011, 14, 1–14.

Hu, Yifan, Mohammad Rostami, and Jesse Thomason, “Expert Personas Improve LLM Alignment but Damage Accuracy,” *arXiv preprint arXiv:2603.18507*, 2026.

IV, Robert L Logan, Ivana Balažević, Eric Wallace, Fabio Petroni, Sameer Singh, and Sebastian Riedel, “Cutting down on prompts and parameters: Simple few-shot learning with language models,” *arXiv preprint arXiv:2106.13353*, 2021.

Kim, Do Yeon, Woo-Jin Lee, Youngho Kim, and Joonsuk Kim, “Persona is a Double-edged Sword,” *arXiv preprint arXiv:2408.08631*, 2024.

Kong, Aobo, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, and Xin Zhou, “Better zero-shot reasoning with role-play prompting,” *arXiv preprint arXiv:2308.07702*, 2023.

Li, Jiahui, Sean Papay, and Roman Klinger, “Are Humans as Brittle as Large Language Models?,” in “Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics” 2025, pp. 2130–2155.

Lu, Yao, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity,” *arXiv preprint arXiv:2104.08786*, 2021.

Lupo, Luca, Måns Magnusson, Dirk Hovy, Elin Naurin, and Lena Wangnerud, “Towards Human-Level Text Coding with LLMs: The Case of Fatherhood Roles in Public Policy Documents,” *arXiv preprint arXiv:2311.11844*, 2024.

McLaren, Lorcan, James P. Cross, Zuzanna Krakowska, Robin Rauner, and Martijn Schoonvelde, “Magic Words or Methodical Work? Challenging Conventional Wisdom in LLM-Based Political Text Annotation,” *arXiv preprint arXiv:2603.26898*, 2026.

Mishra, Swaroop, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi, “Cross-task generalization via natural language crowdsourcing instructions,” *arXiv preprint arXiv:2104.08773*, 2021.

—, —, —, **Yejin Choi, and Hannaneh Hajishirzi**, “Reframing Instructional Prompts to GPTk’s Language,” *arXiv preprint arXiv:2109.07830*, 2021.

Ornstein, Joseph T, Eliot N Blasingame, and Jake S Truscott, “How to Train Your Stochastic Parrot: Large Language Models for Political Texts,” *Political Science Research and Methods*, 2025.

Pan, Jane, Tianyu Gao, Howard Chen, and Danqi Chen, “What In-Context Learning” Learns” In-Context: Disentangling Task Recognition and Task Learning,” *arXiv preprint arXiv:2305.09731*, 2023.

Pecher, Branislav, Marek Spiegel, Jakub Belanec, and Jan Cegin, “Revisiting Prompt Sensitivity in Large Language Models for Text Classification: The Role of Prompt Underspecification,” *arXiv preprint arXiv:2602.04297*, 2026.

Peskine, Youri, Damir Korenčić, Ivan Grubisic, Paolo Papotti, Raphael Troncy, and Paolo Rosso, “Definitions Matter: Guiding GPT for Multi-label Classification,” in “Findings of the Association for Computational Linguistics: EMNLP 2023” 2023, pp. 4054–4063.

Reich, Andreas, Claudia Thoms, and Tobias Schrimpf, “Introducing HALC: A General Pipeline for Finding Optimal Prompting Strategies for Automated Coding with LLMs in the Computational Social Sciences,” *arXiv preprint arXiv:2507.21831*, 2025.

Reiss, Michael V, “Testing the reliability of chatgpt for text annotation and classification: A cautionary remark,” *arXiv preprint arXiv:2304.11085*, 2023.

Reynolds, Laria and Kyle McDonell, “Prompt programming for large language

models: Beyond the few-shot paradigm,” in “Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems” 2021, pp. 1–7.

Ruckdeschel, Mattes, “Just Read the Codebook! Make Use of Quality Codebooks in Zero-Shot Classification of Multilabel Frame Datasets,” in “Proceedings of the 31st International Conference on Computational Linguistics (COLING)” 2025, pp. 6317–6337.

Salewski, Leonard, Stephan Alaniz, Isabel Rio-Torto, Eric Schulz, and Zeynep Akata, “In-Context Impersonation Reveals Large Language Models’ Strengths and Biases,” *Advances in Neural Information Processing Systems*, 2024, 36.

Savelka, Jaromir, Kevin D Ashley, Morgan A Gray, Hannes Westermann, and Huihui Xu, “Can gpt-4 support analysis of textual data in tasks requiring highly specialized domain expertise?,” *arXiv preprint arXiv:2306.13906*, 2023.

Schick, Timo and Hinrich Schütze, “It’s not just size that matters: Small language models are also few-shot learners,” *arXiv preprint arXiv:2009.07118*, 2020.

Sclar, Melanie, Yejin Choi, Yulia Tsvetkov, and Alane Suhr, “Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting,” in “International Conference on Learning Representations (ICLR)” 2024.

Stuhler, Oscar, Adrien Ton, and Étienne Ollion, “From Codebooks to Promptbooks,” *Sociological Methods & Research*, 2025, 54, 794–848.

Tam, Zhi Rui, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung yi Lee, and Yun-Nung Chen, “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models,” in “Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track” 2024.

Törnberg, Petter, “Best Practices for Text Annotation with Large Language Models,” *Sociologica*, 2024, 18 (2), 67–85.

van Elten, Jonas and Stefan P Penczynski, “Coordination games with asymmetric payoffs: An experimental study with intra-group communication,” *Journal of Economic Behavior & Organization*, 2020, 169, 158–188.

Wang, Shucheng and Heather Wang, “Assessing Consistency and Reproducibility in the Outputs of Large Language Models,” *arXiv preprint arXiv:2503.16974*, 2025.

Wang, Xiaolei, Xinyu Tang, Wayne Xin Zhao, and Ji-Rong Wen, “Investigating the Pre-Training Dynamics of In-Context Learning: Task Recognition vs. Task Learning,” *arXiv preprint arXiv:2406.14022*, 2024.

White, Jules, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt, “A prompt pattern catalog to enhance prompt engineering with chatgpt,” *arXiv preprint arXiv:2302.11382*, 2023.

Wilson, Edwin B., “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, 1927, 22 (158), 209–212.

Yuan, J, P Bao, Z Chen, M Yuan, J Zhao, J Pan, Y Xie, Y Cao, Y Wang, Z Wang et al., “Advanced prompting as a catalyst: Empowering large language models in the management of gastrointestinal cancers,” *The Innovation*, 2023.

Yuan, Zhenyuan, Jiawei Jia, Fei Wang, Jiaxin Chen, Wotao Yin, and Tianyi Cai, “Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference,” *arXiv preprint arXiv:2506.09501*, 2025. NeurIPS 2025 Oral.

Zhang, Xing, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He, “Prompt Optimization Is a Coin Flip,” *arXiv preprint arXiv:2604.14585*, 2026.

Zhao, Wayne Xin, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong et al., “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.

Zheng, Mingqian, Jiaxin Pei, and David Jurgens, “When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models,” in “Findings of the Association for Computational Linguistics: EMNLP 2024” 2024.

Ziems, Caleb, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang, “Can large language models transform computational social science?,” *Computational Linguistics*, 2024, pp. 1–55.

A. Content Ablation

A.1. Information surfaces

Figure 4 presents L_{II} information surfaces for all four models. Accuracy increases monotonically with both experiment context and theory context, and the gain from detailed classification instructions is visible as a shift between the left (simple) and right (verbatim) panels. GPT 5.4 and Qwen 397B show relatively flat surfaces on L_I (Figure 2), indicating robustness to context removal; GPT 5.4-nano and Qwen 9B show steeper gradients, reflecting greater dependence on prompt detail. On L_{II} , all models show steeper gradients, consistent with the greater role of theory context for this task.

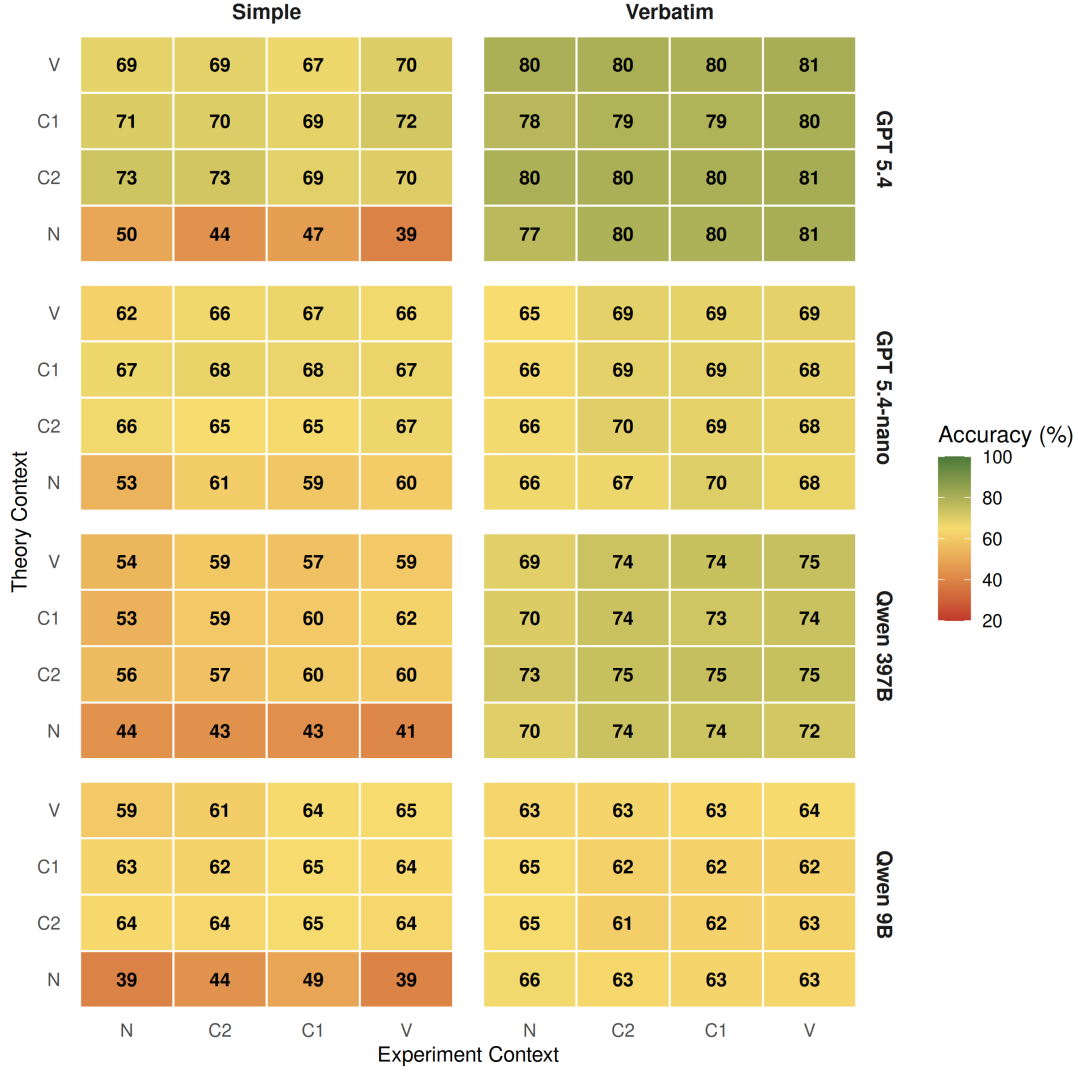


Figure 4: L_{II} information surface (all four models). Accuracy by experiment context (rows) and theory context (columns) under simple (left) and verbatim (right) classification instructions.

A.2. Variance decomposition

	P	L_I	L_{II}
Classification instructions	1.0%	58.3%	27.4%
Theory context	—	8.5%	9.6%
Experiment context	1.1%	1.3%	1.9%
Model	97.8%	31.9%	61.1%

Note: Unique explained variance, Type III likelihood-ratio tests. Each row: unique contribution after accounting for all others.

Table 8: Variance explained by component.

Table 9 reports the per-model variance decomposition using the same Type III likelihood-ratio methodology. The pattern confirms that model scale determines which prompt component dominates: for GPT 5.4, experiment context explains 81.5% of variance on P (classification instructions only 18.5%), whereas GPT 5.4-nano’s variance on L_I is almost entirely driven by classification instructions (99.9%).

Model	P		L_I			L_{II}		
	Cls	Exp	Cls	Theory	Exp	Cls	Theory	Exp
GPT 5.4	18.5	81.5	57.4	41.6	0.9	69.7	30.2	0.1
GPT 5.4-nano	57.6	42.4	99.9	0.1	0.0	39.2	40.7	20.1
Qwen 397B	5.9	94.1	70.3	16.3	13.4	85.5	12.2	2.3
Qwen 9B	0.0	100.0	97.9	1.2	0.9	26.1	73.8	0.0

Note: Unique explained variance (%), Type III likelihood-ratio tests within each model. P has no theory dimension. Rows may not sum to 100 due to shared variance.

Table 9: Per-model variance decomposition (%).

A.3. Full cosine similarity table

Table 10 reports the cosine similarity between each compressed prompt variant and its verbatim source. For each text, we submit the full content as a single input to OpenAI’s `text-embedding-3-large` model (via the Azure OpenAI API) and obtain a 3072-dimensional embedding vector; the reported similarity is the cosine of the angle between the verbatim and compressed vectors. Cosine similarity ranges from -1 (opposite) to 1 (identical), with 0 indicating orthogonal vectors; for embeddings of related natural-language texts, values above ~ 0.8 indicate that the two texts encode

highly overlapping semantic content. Compression ratios show the fraction of verbatim word count retained.

Task	Source	Level	Cosine	Words	Compression
P	Exp. context	V	1.000	108	100%
P	Exp. context	C1	0.978	63	58%
P	Exp. context	C2	0.866	33	31%
P	Cls. instructions	V	1.000	—	100%
P	Cls. instructions	S	0.772	—	11%
L_I	Exp. context	V	1.000	293	100%
L_I	Exp. context	C1	0.857	174	59%
L_I	Exp. context	C2	0.831	97	33%
L_I	Theory	V	1.000	735	100%
L_I	Theory	C1	0.901	374	51%
L_I	Theory	C2	0.804	158	21%
L_I	Cls. instructions	V	1.000	—	100%
L_I	Cls. instructions	S	0.724	—	3%
L_{II}	Exp. context	V	1.000	732	100%
L_{II}	Exp. context	C1	0.971	362	49%
L_{II}	Exp. context	C2	0.940	226	31%
L_{II}	Theory	V	1.000	362	100%
L_{II}	Theory	C1	0.948	198	55%
L_{II}	Theory	C2	0.839	94	26%
L_{II}	Cls. instructions	V	1.000	—	100%
L_{II}	Cls. instructions	S	0.722	—	5%

Note: Minimal (N) removes the section entirely (0% compression). Classification instruction word counts omitted as they vary by task structure. Embeddings computed with Azure OpenAI `text-embedding-3-large` (3072 dimensions).

Table 10: Cosine similarity and compression ratio for all prompt variants.

B. Format and Framing

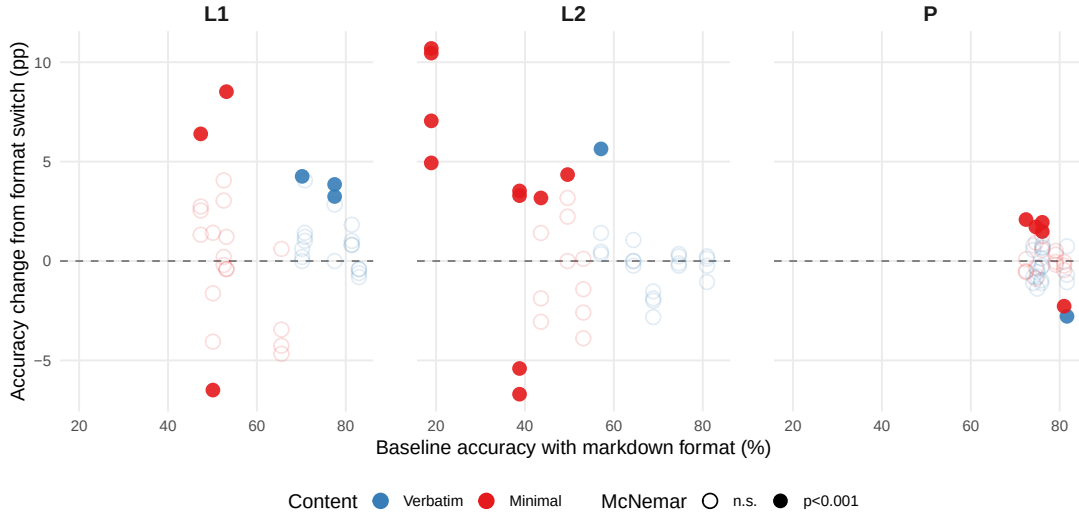


Figure 5: Format effects by task.

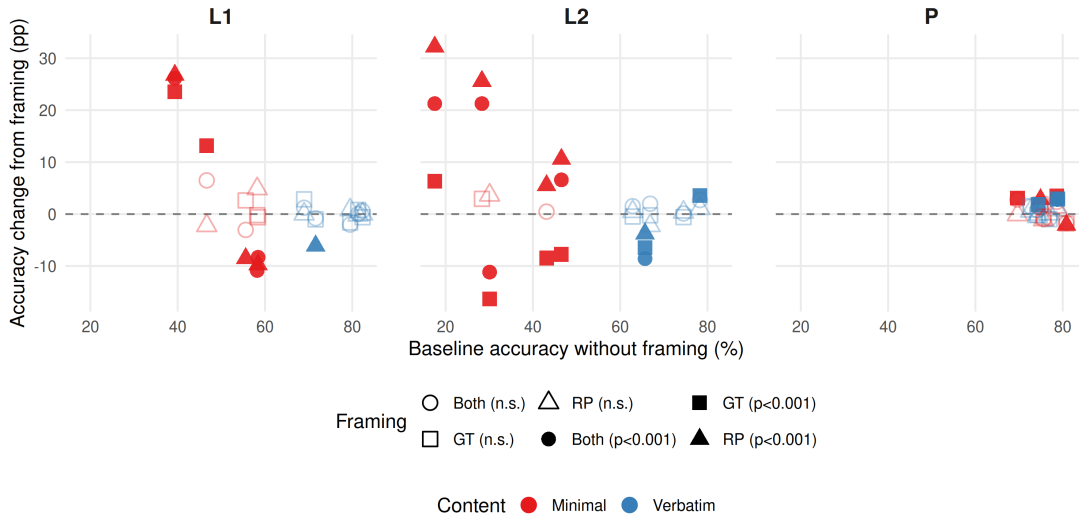


Figure 6: Framing effects by task.

Table 11 reports the mean accuracy change ($\bar{\Delta}$, pp) and its standard deviation (σ) across format and framing treatments relative to the baseline. Format cells pool 12 comparisons each (4 alternative formats $\times$ 3 tasks); framing cells pool 9 comparisons each (3 framing conditions $\times$ 3 tasks, vs. the no-framing baseline).

Table 11: Format and framing: mean $\bar{\Delta}$ (pp) and σ vs. baseline.

Model	Format				Framing			
	Minimal		Verbatim		Minimal		Verbatim	
	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ
GPT 5.4	+1.5	2.5	−0.2	1.6	+6.0	10.9	+1.9	1.7
GPT 5.4-nano	+0.2	2.3	−0.6	1.8	+1.0	6.4	+0.4	1.5
Qwen 397B	−0.9	2.7	+0.1	0.7	−3.2	4.9	−0.3	0.7
Qwen 9B	+0.5	4.0	+0.6	1.4	+7.9	12.3	−0.2	2.5

B.1. Framing significance — all models

Table 12 extends the framing results (Table 4) by reporting the number of significant effects by framing treatment. On verbatim prompts, only GPT 5.4 shows any appreciable framing effects (3 of 9 comparisons, all positive); Qwen 9B has one significant negative effect, and the other two models show none. On minimal prompts, framing effects are more common but inconsistent in direction: GPT 5.4-nano and Qwen 397B both reach 5 of 9, but Qwen 397B’s effects are predominantly negative (4 of 5) while Qwen 9B’s are all positive (4 of 4).

	Verbatim				Minimal			
	Total	+	−	% neg	Total	+	−	% neg
GPT 5.4	3	3	0	0%	4	3	1	25%
GPT 5.4-nano	0	0	0	—	5	3	2	40%
Qwen 397B	0	0	0	—	5	1	4	80%
Qwen 9B	1	0	1	100%	4	4	0	0%

Note: Number of McNemar comparisons reaching $p < 0.001$ out of 9 per cell (3 framing treatments $\times$ 3 tasks, vs. no-framing baseline). +/− = direction of significant effect.

Table 12: Framing: number of significant effects by model and content level.

Figure 7 shows per-model framing effects at both content levels. Each point represents one task $\times$ framing treatment comparison against the no-framing baseline; filled shapes indicate significance at $p < 0.001$. Error bars show ± 1 standard deviation of the effects within each content level.

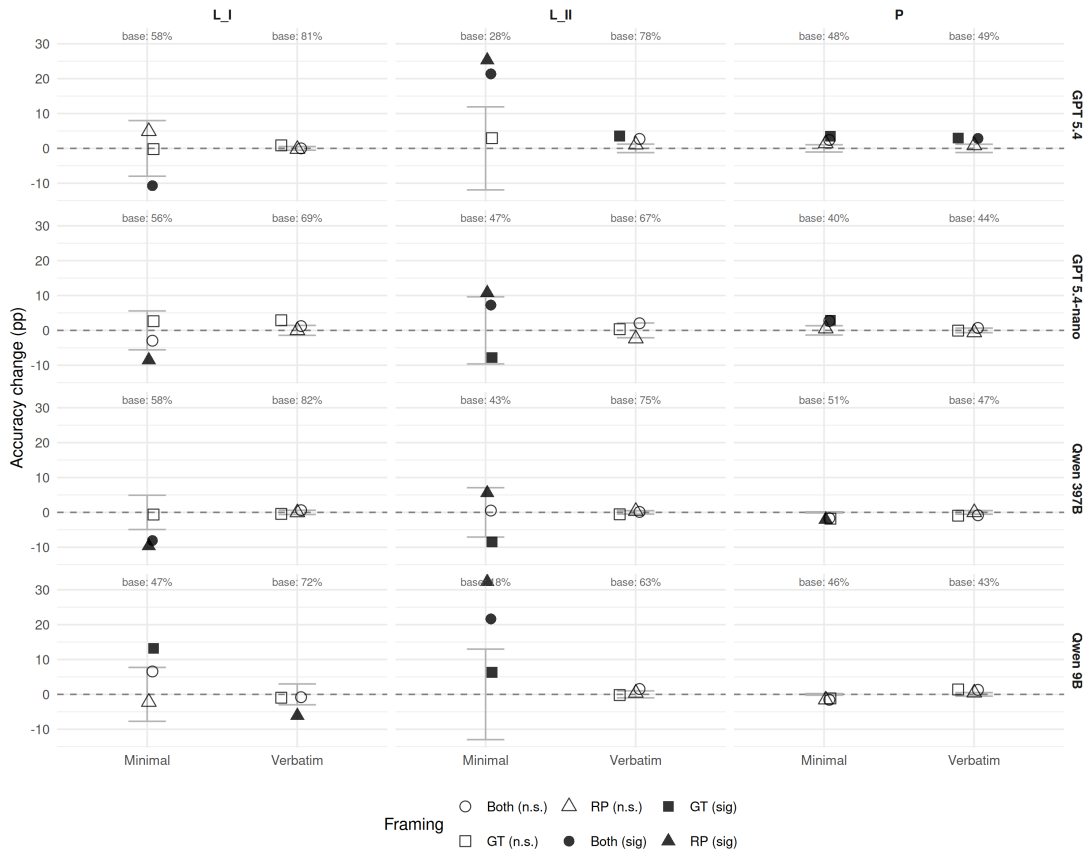


Figure 7: Per-model framing effects. Shapes encode framing type (GT only, RP only, Both) and significance. Baseline accuracy annotated at the top of each panel.

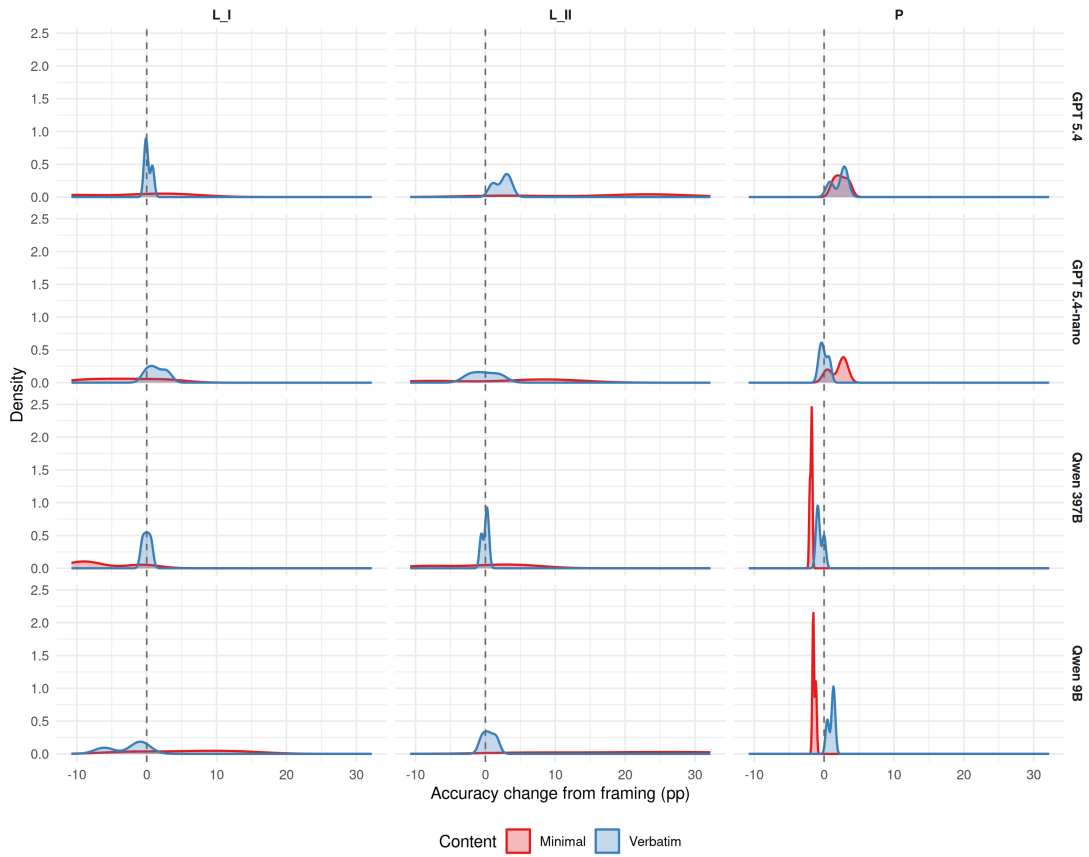


Figure 8: Distribution of framing effects by task and model. Red = minimal content; blue = verbatim content.

C. Examples

C.1. Example counts by label

Task	Label	Examples
P	Promise (1)	11
P	Empty talk (0)	10
L_I	Level 0	3
L_I	Level 1	2
L_I	Level 2	6
L_I	Level 3	2
L_{II}	Level 0	2
L_{II}	Level 1	2
L_{II}	Level 2	1
L_{II}	Level 3	0

Note: Counts reflect the worked examples available in each codebook. P : 21 total (10.5 per label). L_I : 13 total (3.25 per level). L_{II} : 5 total (1.25 per level); level 3 is unrepresented.

Table 13: Number of worked examples per classification label.

C.2. Content–examples substitution (all models)

Tables 14–16 report the same decomposition as Table 5 for the remaining three models.

	P	L_I	L_{II}
Δ_{Content}	+3.3	+17.7*	+15.8*
Δ_{Ex}	+9.1*	+20.1*	+10.8*
$\Delta_{\text{Ex} \text{Content}}$	+7.7*	+8.1*	+1.8
$\Delta_{\text{Content} \text{Ex}}$	+1.9	+5.7	+6.8*

Table 14: GPT 5.4-nano: content–examples decomposition.

	P	L_I	L_{II}
Δ_{Content}	−4.3*	+32.9*	+31.0*
Δ_{Ex}	+16.5*	+32.1*	+25.3*
$\Delta_{\text{Ex} \text{Content}}$	+20.5*	+0.6	+3.2
$\Delta_{\text{Content} \text{Ex}}$	−0.3	+1.4	+8.9*

Table 15: Qwen 397B: content–examples decomposition.

	P	L_I	L_{II}
Δ_{Content}	-0.9	+17.7*	+25.6*
Δ_{Ex}	+14.2*	+12.4*	+30.6*
$\Delta_{\text{Ex} \text{Content}}$	+15.3*	+12.0*	+3.8
$\Delta_{\text{Content} \text{Ex}}$	+0.2	+17.3*	-1.2

Table 16: Qwen 9B: content-examples decomposition.

C.3. Incremental effects — all models

Tables 17 and 18 report the accuracy at four key conditions for all four models: minimal 0-shot (baseline), minimal n -shot (+examples), verbatim 0-shot (+content), and verbatim n -shot (+both).

	Minimal		Verbatim	
	0-shot	n -shot	0-shot	n -shot
GPT 5.4	73.2	89.8	74.2	88.2
GPT 5.4-nano	60.9	70.0	64.3	72.0
Qwen 397B	70.5	87.0	66.2	86.7
Qwen 9B	64.1	78.3	63.3	78.5

Note: No reasoning. Wilson 95% CIs omitted for space.

Table 17: P : Accuracy (%) at four key conditions, all models.

	L_I				L_{II}			
	Minimal		Verbatim		Minimal		Verbatim	
	0-shot	n -shot	0-shot	n -shot	0-shot	n -shot	0-shot	n -shot
GPT 5.4	47.4	84.8	81.3	84.0	49.6	70.0	81.0	81.7
GPT 5.4-nano	52.5	72.6	70.2	78.3	53.1	63.9	68.9	70.7
Qwen 397B	50.1	82.2	83.0	83.6	43.6	68.9	74.6	77.8
Qwen 9B	53.1	65.5	70.8	82.8	38.8	69.4	64.4	68.2

Note: No reasoning. Wilson 95% CIs omitted for space.

Table 18: L_I and L_{II} : Accuracy (%) at four key conditions, all models.

C.4. Example Format

As with prompt formatting (Section 6.2), we also tested whether how examples are presented within the prompt matters. We compared five presentation styles — markdown headers, XML tags, plain delimiters, arrow notation, and multi-turn conversation — at both content levels. In the first four styles, examples are embedded in the system message alongside the codebook content. In the multi-turn style, each example is sent as a separate user–assistant exchange before the classification query.

Table 19 reports the proportion of significant effects when switching away from the markdown baseline to alternative example formats.

	In-prompt formats				Multi-turn			
	Minimal		Verbatim		Minimal		Verbatim	
	+	−	+	−	+	−	+	−
GPT 5.4	.00	.00	.00	.00	.00	.33	.00	.33
GPT 5.4-nano	.00	.00	.00	.00	.00	.67	.00	.00
Qwen 397B	.00	.33	.00	.33	.00	.67	.00	.33
Qwen 9B	.22	.33	.00	.33	.00	.67	.00	.33

Note: Proportion of all comparisons that are significantly positive (+) or negative (−) at $p < 0.001$, vs. markdown baseline. In-prompt: 9 per cell (3 formats $\times$ 3 tasks). Multi-turn: 3 per cell (1 format $\times$ 3 tasks).

Table 19: Example format: proportion of significant effects.

For GPT models, in-prompt format variation produces zero significant effects; only multi-turn is harmful. Qwen models are more sensitive: even in-prompt format switches produce up to .33 significant negatives, and multi-turn adds further damage (.67 on minimal). Overall, switching between in-prompt styles produces no significant effect for GPT models and only scattered effects for Qwen models. The one exception is multi-turn presentation: placing examples outside the system message as conversation history consistently hurts all models, particularly on minimal prompts.

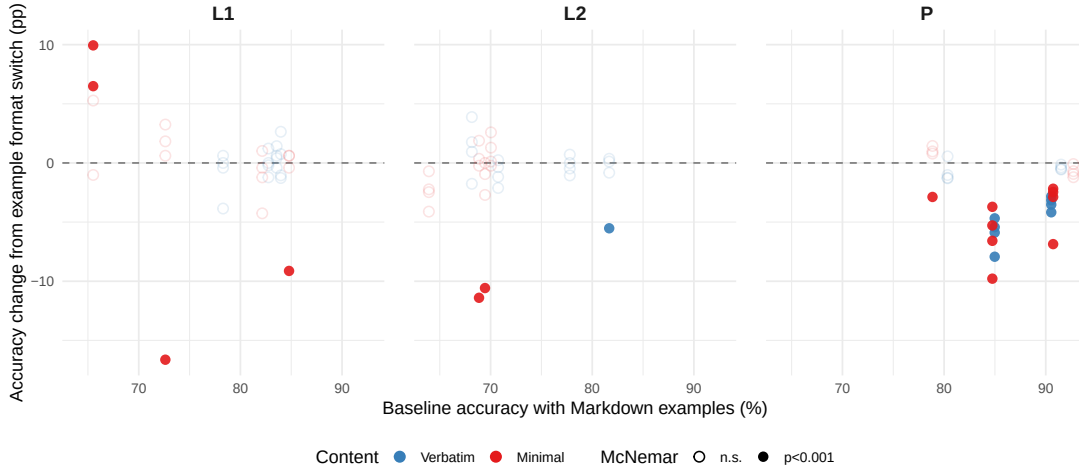


Figure 9: Example format effects by task.

Table 20 summarises the mean accuracy change ($\bar{\Delta}$, pp) and its standard deviation (σ) when switching away from the markdown baseline to alternative example formats. In-prompt formats (XML, delimiter, arrow) pool 9 comparisons per cell (3 alternative formats $\times$ 3 tasks); multi-turn pools 3 comparisons (1 format $\times$ 3 tasks).

Table 20: Example format: mean $\bar{\Delta}$ (pp) and σ vs. markdown baseline.

Model	In-prompt (9)				Multi-turn (3)			
	Minimal		Verbatim		Minimal		Verbatim	
	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ
GPT 5.4	-0.2	1.0	-0.4	0.6	-2.2	6.2	-1.2	4.1
GPT 5.4-nano	+0.2	2.5	-0.4	0.9	-7.1	8.4	-2.6	1.1
Qwen 397B	-1.0	2.1	-1.3	2.5	-8.5	3.7	-2.2	3.3
Qwen 9B	-1.2	6.3	-2.4	4.1	-6.4	10.3	-2.5	7.7

D. Reasoning

D.1. Reasoning dosage — GPT models

Table 21 reports the incremental effect of each reasoning step on verbatim content for GPT 5.4 and GPT 5.4-nano. For GPT 5.4, the first step (off $\rightarrow$ low) produces significant gains at 0-shot, but every subsequent step is non-significant or significantly negative. At n -shot, even the first step is marginal. GPT 5.4-nano benefits more from the initial step, and shows significant gains from extra-high reasoning on L_{II} at both

shot conditions — consistent with smaller models extracting more from extended reasoning.

	Step	0-shot			n -shot		
		P	L_I	L_{II}	P	L_I	L_{II}
GPT 5.4	off $\rightarrow$ low	+4.6*	+1.6	-1.9	+0.8	+0.6	-2.0
	low $\rightarrow$ med	-0.1	+1.0	+2.4	+0.5	+2.0	+1.9
	med $\rightarrow$ high	-5.7*	-1.4	-0.5	+0.0	-0.6	+2.0
	high $\rightarrow$ xhigh	-5.6*	+0.0	+0.7	-0.3	-1.4	-0.7
GPT 5.4-nano	off $\rightarrow$ low	+5.2*	+1.6	-1.1	+5.8*	+0.6	-4.1*
	low $\rightarrow$ med	+1.5	+5.1	+0.2	+4.1*	+0.6	+3.4
	med $\rightarrow$ high	-0.2	-1.2	+2.5	+1.1	-0.2	+0.4
	high $\rightarrow$ xhigh	-2.0	+4.7	+6.1*	+2.9*	+3.7	+8.3*

Note: * = McNemar $p < 0.001$. All values in pp, each step relative to the preceding level. Reasoning token counts: low ≈ 113 , med ≈ 309 , high ≈ 682 , xhigh $\approx 2,479$. Table 22 reports high reasoning to match Qwen’s reasoning token output (≈ 750 tokens).

Table 21: Reasoning dosage: incremental step effects on verbatim content.

D.2. Effect of reasoning on verbatim content

Table 22 reports the effect of enabling reasoning on verbatim content at both zero-shot and n -shot for all four models.

	0-shot			n -shot		
	P	L_I	L_{II}	P	L_I	L_{II}
GPT 5.4	-1.1	+1.2	+0.0	+1.3	+2.0	+1.9
GPT 5.4-nano	+6.5*	+5.5	+1.6	+11.1*	+1.0	-0.4
Qwen 397B	+8.4*	+1.0	+0.6	+3.1*	+2.8	+2.6
Qwen 9B	+9.4*	+10.5*	+2.4	+8.7*	+1.6	+4.0

Note: * = McNemar $p < 0.001$. All values in pp: accuracy with reasoning minus accuracy without, on verbatim content. GPT: high reasoning (≈ 682 tokens); Qwen: reasoning on (≈ 780 tokens).

Table 22: Effect of reasoning on verbatim content.

GPT 5.4 shows no significant effect on any task at any condition. Smaller models benefit selectively: on P , reasoning helps all three non-frontier models significantly at both 0-shot and n -shot, with the largest gains for Qwen 9B (+9.4 and +8.7 pp). On L_I ,

Qwen 9B gains +10.5 pp at zero-shot, but this drops to +1.6 pp (n.s.) once examples are present. On L_{II} , no model shows a significant reasoning effect at either condition.

D.3. Content vs. reasoning substitutability (all models)

Tables 23–25 extend Table 7 to the remaining three models. The pattern is consistent: content retains its full value when reasoning is active ($\Delta_{\text{Content}|\text{Rea}}$ is large on L -tasks), while reasoning given content ($\Delta_{\text{Rea}|\text{Content}}$) is near-zero on L -tasks for all models except Qwen 9B on L_I . On P , smaller models show significant reasoning-given-content effects, consistent with reasoning partially compensating for weaker pre-training representations.

	P	L_I	L_{II}
Δ_{Rea}	+9.7*	+11.2*	−9.5*
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+6.4*	−6.5	−25.3*
$\Delta_{\text{Content} \text{Rea}}$	+0.1	+12.0*	+26.9*
$\Delta_{\text{Rea} \text{Content}}$	+6.5*	+5.5	+1.6

Table 23: GPT 5.4-nano: content vs. reasoning substitutability.

	P	L_I	L_{II}
Δ_{Rea}	+0.1	+5.3	−1.6
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+4.4*	−27.6*	−32.7*
$\Delta_{\text{Content} \text{Rea}}$	+3.9*	+28.6*	+33.2*
$\Delta_{\text{Rea} \text{Content}}$	+8.4*	+1.0	+0.6

Table 24: Qwen 397B: content vs. reasoning substitutability.

	P	L_I	L_{II}
Δ_{Rea}	+2.9	+5.4	−11.6*
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+3.7	−12.3*	−37.2*
$\Delta_{\text{Content} \text{Rea}}$	+5.7*	+22.8*	+39.6*
$\Delta_{\text{Rea} \text{Content}}$	+9.4*	+10.5*	+2.4

Table 25: Qwen 9B: content vs. reasoning substitutability.

E. Information Fidelity

E.1. Verbosity and noise

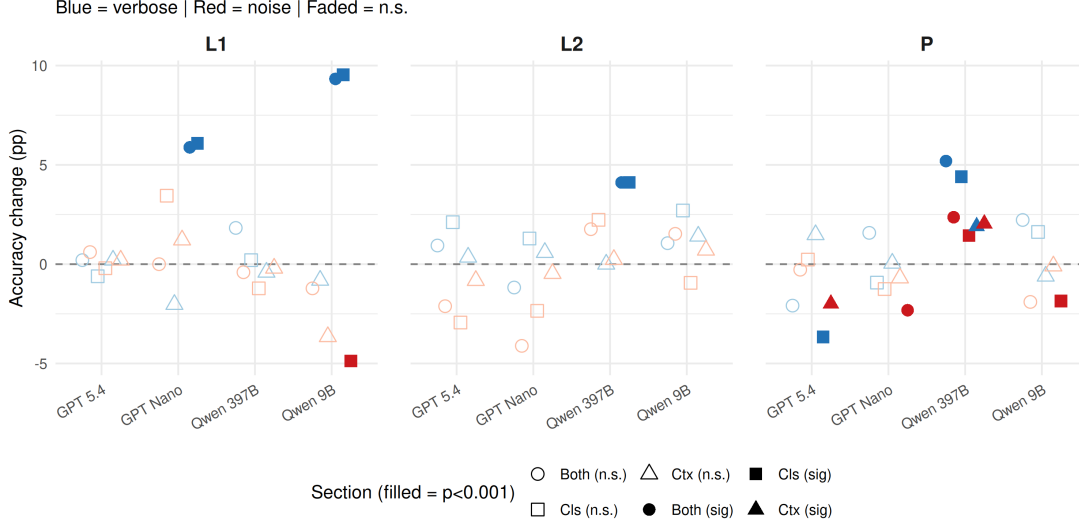


Figure 10: Verbosity and noise effects (0-shot).

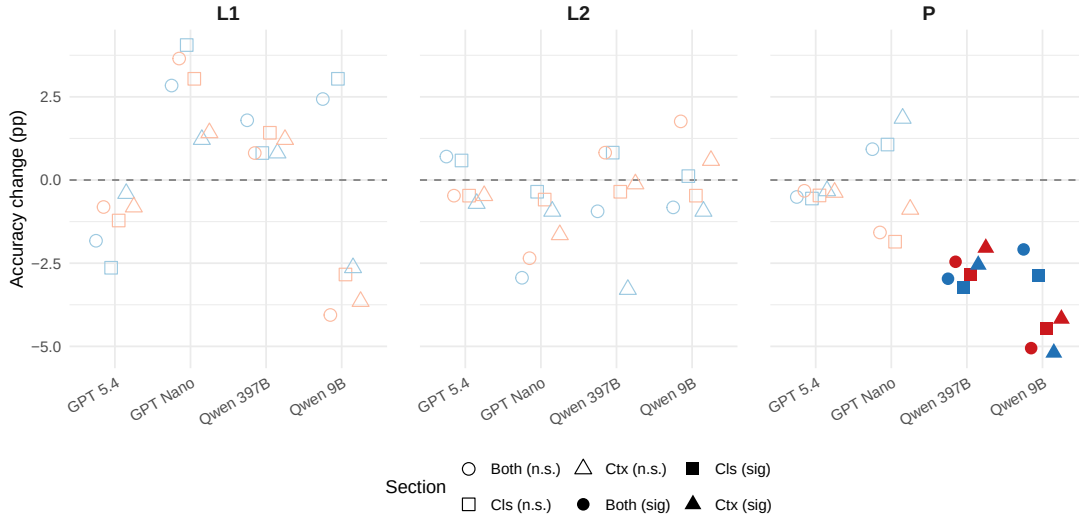


Figure 11: Verbosity and noise effects (n-shot).

Table 26 reports the mean accuracy change ($\bar{\Delta}$, pp) and its standard deviation (σ) for verbosity and noise treatments relative to the baseline. Each cell pools 9 comparisons (3 levels $\times$ 3 tasks).

Table 26: Verbosity and noise: mean $\bar{\Delta}$ (pp) and σ vs. baseline.

Model	0-shot				n-shot			
	Verbose		Noise		Verbose		Noise	
	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ	$\bar{\Delta}$	σ
GPT 5.4	−0.3	2.4	−0.9	1.4	−0.7	1.0	−0.7	0.3
GPT 5.4-nano	+1.3	3.0	−0.9	2.3	+1.1	2.2	−0.2	2.4
Qwen 397B	+2.9	2.8	+1.2	1.6	−1.4	2.6	−0.7	2.2
Qwen 9B	+3.1	3.8	−1.5	2.1	−1.5	3.2	−3.1	3.1

E.2. Paraphrasing

Each content section of the verbatim prompt is independently translated to a target language (Turkish or German) and back to English using an LLM, producing a semantically equivalent text with altered wording and syntax. Cosine similarity to the original remains 0.97–0.99; token counts change by $\pm 5\%$. This makes wording the primary source of variation. Each language pair is tested on all 3 tasks at both 0-shot and n -shot, yielding 6 comparisons per model per shot condition (24 total per model).

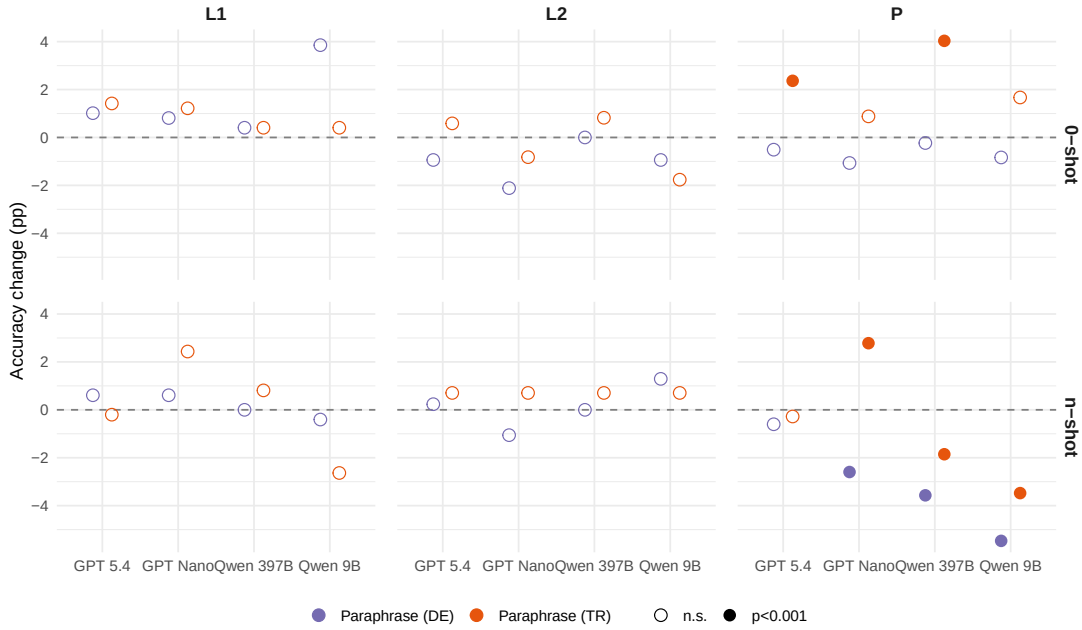


Figure 12: Paraphrase effects by task and model. Top: 0-shot. Bottom: n -shot. Orange = Turkish back-translation; purple = German back-translation. Filled = $p < 0.001$.

At 0-shot (Figure 12, top), effects are negligible: the median absolute change is

below 1 pp for all models, and GPT 5.4 shows zero significant effects across all 6 comparisons. L_{II} is entirely unaffected for every model. The direction of non-significant changes is evenly split — positive and negative with no pattern.

At n -shot (Figure 12, bottom), a few significant effects emerge, concentrated on P for smaller models. GPT 5.4 remains unaffected. The significant n -shot effects are predominantly negative, suggesting that paraphrasing occasionally disrupts the alignment between example wording and classification criteria — but only for models that are already sensitive to surface variation, and only when examples are present to create a wording mismatch.

F. Full Experiment Designs

F.1. Content Ablation

Each prompt is assembled from three codebook sections: experiment context, theory context (L -tasks only), and classification instructions. Each section is varied independently across compression levels:

Section	Levels	Description
Experiment context	V, C1, C2, N	Verbatim, $\approx 50\%$, $\approx 25\%$, removed
Theory context	V, C1, C2, N	Verbatim, $\approx 50\%$, $\approx 25\%$, removed
Classification instructions	V, S	Verbatim, simplified (1-line)

P -tasks have no theory dimension, yielding $4 \times 2 = 8$ variants per task. L -tasks cross all three dimensions: $4 \times 4 \times 2 = 32$ variants per task. Total: 80 variants per model ($8 + 8 + 32 + 32$).

Compressions are generated iteratively ($V \rightarrow C1$, $C1 \rightarrow C2$) using the structured compression prompt in Appendix J.2, which instructs the model to only remove content while preserving a task-specific list of key technical terms. Cosine similarity to the verbatim text remains above 0.80 at all levels (Table 2; full results for all tasks in Appendix A.3). The filtering prompt used to prepare the verbatim text from the raw codebook is in Appendix J.1. Figure 13 illustrates the result on the L_I experiment context.

Verbatim (V) — 293 words

- A single experimental session consists of 4 parts and every part consists of 2 periods. In every part, every subject is randomly paired with another one to form a team.
- Depending on the treatment, every team then is randomly matched with 2 or 5 other teams to form a voting group.
- Every group will be assigned to a blue or red urn and every team in the group will draw a ball from their assigned urn. The blue urn contains twice more blue balls than red balls. The red urn contains twice more red balls than blue balls.
- [... five further statements: group size, draw mechanism, voting rule, message protocol, and final decision.]

C1 ($\approx 50\%$) — 174 words

- Every subject is randomly paired with another one to form a team. Every team is randomly matched with other teams to form a voting group.
- Every group will be assigned to a blue or red urn and every team in the group will draw a ball from their assigned urn. The blue urn contains twice more blue balls than red balls. The red urn contains twice more red balls than blue balls.
- The groups consist of either 3 or 6 teams. The balls will be drawn from the urn with or without replacement.
- [...]

C2 ($\approx 25\%$) — 97 words

- Subjects form teams of two. Teams are grouped into voting groups of 3 or 6 teams.
- Every group is assigned to a blue or red urn. The blue urn contains twice more blue balls than red balls, and vice versa. Every team draws a ball, with or without replacement.
- The goal is to correctly state the color of the urn. Every team submits a vote (red or blue); if every team votes red, the group decision is red, otherwise blue.
- Each subject sends a message to their team member explaining their suggested vote.

Figure 13: The L_1 experiment context at three compression levels. Each level is produced from the previous one with the compression prompt in Appendix J.2. The C2 statements are given in full; V and C1 are truncated for space.

F.2. Format

Five visual formats decomposed into three binary features (markdown markers, section titles, bullet lists):

Format	Md markers	Titles	Bullets	Description
ml (baseline)	✓	✓	✓	# headers + - bullets
tl		✓	✓	Plain titles + - bullets
mp		✓		Plain titles + prose paragraphs
fl			✓	No headers + - bullets
pt				No headers + prose paragraphs

Each format is tested at two content levels (verbatim and minimal), yielding $4 \times 2 = 8$ new variants per task (the ml baseline is shared with Experiment A).

The following example shows the same classification instruction content rendered in each format:

```

m1 (markdown + list)
## Level 0
- Chooses randomly, without justification or through some
justification completely unrelated to the task.
- Might not have understood the game or shows no interest in it.

t1 (titles + list)
Level 0
- Chooses randomly, without justification or through some
justification completely unrelated to the task.
- Might not have understood the game or shows no interest in it.

mp (titles + prose)
Level 0
Chooses randomly, without justification or through some
justification completely unrelated to the task. Might not have
understood the game or shows no interest in it.

f1 (flat list)
[header removed]
- Chooses randomly, without justification or through some
justification completely unrelated to the task.
- Might not have understood the game or shows no interest in it.

pt (plain text)
[header removed]
Chooses randomly, without justification or through some
justification completely unrelated to the task. Might not have
understood the game or shows no interest in it.

```

Figure 14: The same classification instruction in all five format treatments.

F.3. Framing

Two framing elements are tested by removal:

Condition	Role persona	General task
full (baseline)	✓	✓
norp		✓
nogt	✓	
none		

Each condition is tested at two content levels (verbatim and minimal), yielding $3 \times 2 = 6$ new variants per task (the “full” baseline is shared with Experiment A). Total: 12 files per model.

The “General Task” and “Role Persona” are the two framing sections of the prompt template (Figure 1); all other sections are held fixed. Figure 15 instantiates them for L_1 and shows the four framing conditions, following the template’s markdown layout.

```

full (baseline) — general task + role persona
# General Task
- Classify a player’s level of strategic thinking (0--3) based on
their message in a voting game experiment.
# Role Persona
- Act as a behavioral economist with expertise in text
classification, bounded rationality, and level-k reasoning.
# Context
- ...

norp — general task only
# General Task
- Classify a player’s level of strategic thinking (0--3) based on
their message in a voting game experiment.
# Context
- ...

nogt — role persona only
# Role Persona
- Act as a behavioral economist with expertise in text
classification, bounded rationality, and level-k reasoning.
# Context
- ...

```

Figure 15: The L_1 prompt under each framing condition, following the template in Figure 1. Only the # General Task and # Role Persona sections are added or removed; all other sections are held fixed.

F.4. Examples and Reasoning

Examples. Five example presentation styles are tested:

Style	Description
n (markdown)	Examples formatted with markdown headers and bullets
nx (XML)	Examples wrapped in <code><example></code> tags
nd (delimiter)	Examples separated by <code>---</code> delimiters
na (arrow)	Input $\rightarrow$ Output arrow format
t (multi-turn)	Examples as user/assistant conversation turns

Each style is crossed with 2 content levels $\times$ 3 tasks (P excluded — no worked

examples in codebook) = 20 files per model.

The following example shows the same worked example (from the P task) rendered in each in-prompt style:

```
n (markdown)
## Example 1
### Message
P1: all 200 then?
P2: yes
### Classification
{'p1': '0', ...}

nx (XML)
<example>
  <message>
    P1: all 200 then?
    P2: yes
  </message>
  <classification>
    {'p1': '0', ...}
  </classification>
</example>

nd (delimiter)
Example 1:
Message: P1: all 200 then?
P2: yes
Classification: {'p1': '0', ...}

na (arrow)
`P1: all 200 then? P2: yes` → {'p1': '0', ...}

t (multi-turn) --- not in system prompt; sent as conversation turns:
User: P1: all 200 then? P2: yes
Assistant: {'p1': '0', ...}
```

Figure 16: The same worked example in all five presentation styles.

Reasoning. GPT models support 4 reasoning effort levels (low, medium, high, extra-high); Qwen models support a binary toggle (off/on). Reasoning is crossed with content (verbatim/minimal) and shot (0-shot/ n -shot), yielding up to 32 additional variants per model for GPT and 8 for Qwen.

F.5. Information Fidelity

Three manipulations test whether LLMs respond to informational content or textual surface.

Verbosity and noise. The verbatim prompt sections are either expanded with relevant elaboration (*verbose*: +50–275% words) or padded with irrelevant text of matched

length (*noise*: red squirrel ecology facts interspersed between original sentences). Each manipulation is applied to three scopes, yielding a 2×3 design:

Condition	Verbose		Noise	
	Context	Cls	Context	Cls
cls only		✓		✓
ctx only	✓		✓	
both	✓	✓	✓	✓

Each of the 6 conditions (3 verbose + 3 noise) is tested at 0-shot and n -shot, yielding 12 variants per task. Unmarked sections retain their verbatim content. The noise control isolates the effect of prompt length from informational content: if verbosity helps because of the *content* added, then matched-length noise should not.

Paraphrasing. Round-trip back-translation (English $\rightarrow$ Turkish $\rightarrow$ English, English $\rightarrow$ German $\rightarrow$ English) preserves semantic content while changing surface wording. Tested at 0-shot and n -shot (14 files per model).

Raw codebook. The unprocessed codebook (with payment instructions, software notes, and formatting artifacts) is used directly as the prompt (1 file per task).

The following example shows the same experiment context passage in all three versions (from the P task):

Normal (verbatim)

- Participants can invest between 0--200 in a group project.
- Participants can keep any money that they don't invest. The total amount invested by all 3 group members is doubled.
- Then it is shared out equally.

Verbose (relevant elaboration, +50--275% words)

- Participants can invest between 0--200 in a group project. This is a continuous choice --- participants can invest any integer amount within this range, not just all-or-nothing.
- Participants can keep any money that they don't invest. The total amount invested by all 3 group members is doubled. Then it is shared out equally. This means that from the individual investor's perspective, investing 100 pence returns only 66 pence to the investor --- a net loss of 34 pence. However, the group collectively gains 200 pence (66×3) from that 100 pence investment.

Noise (irrelevant padding, matched word count)

- Participants can invest between 0--200 in a group project.
- An adult red squirrel weighs between 250--340 grams and has a body length of 19--23 cm plus a tail of 15--20 cm.
- Participants can keep any money that they don't invest. The total amount invested by all 3 group members is doubled.
- Red squirrels build nests called dreys, typically located 6--12 meters above ground, using twigs, moss, and stripped bark.
- Then it is shared out equally.

Figure 17: The same experiment context in three versions: normal, verbose (relevant elaboration), and noise (irrelevant padding matched in word count).

G. Cross-Generation Agreement

Table 27 reports Krippendorff's α across three GPT generations (4o, 4.1, 5.4) on identical items using the verbatim n -shot prompt, treating each generation as a rater.

	P	L_I	L_{II}
4o - 4.1	.91	.88	.71
4.1 - 5.4	.89	.84	.66
4o - 5.4	.87	.81	.56
3-way	.89	.84	.63

Note: Krippendorff's α (nominal) on verbatim n -shot prompts. Each GPT generation treated as a rater.

Table 27: Cross-generation agreement (Krippendorff's α).

Adjacent generations agree well: $\alpha = 0.91$ (4o-4.1) and 0.89 (4.1-5.4) on P , 0.88

and 0.84 on L_I . Agreement decays with generational distance — 4o–5.4 is consistently the lowest pair — and with task difficulty: L_{II} reaches only $\alpha = 0.56$ –0.71. Since accuracy improves monotonically across generations, the disagreement reflects improvement rather than drift: a newer model from the same family can be expected to produce results at least as reliable as substituting one trained human annotator for another.

H. API and Implementation Details

All classifications are performed via API calls to four providers. Table 28 summarises the configuration per pool.

Pool	Provider	Models	Endpoints
Azure OpenAI	Microsoft	GPT 5.4 family	7
Together AI	Together	Qwen 3.5 9B	1
OpenRouter	Alibaba	Qwen 3.5 397B/122B/35B/27B	1

Table 28: API pools.

Temperature and determinism. Temperature is set to 0 for all non-reasoning runs, maximising output determinism. Reasoning runs require temperature 1 (a constraint imposed by GPT models; applied uniformly across all providers for consistency). A fixed seed (31) is set for all calls where the API supports it.

Structured output. All calls enforce structured JSON output via a task-specific JSON schema passed in the `response_format` parameter. This ensures that every response is machine-parseable without post-processing.

Logprobs. Token-level log-probabilities are collected for all non-reasoning runs (`top_logprobs = 5` for GPT, Together, and OpenRouter; 10 for Foundry). Logprobs are disabled for reasoning runs, as GPT does not support them when reasoning is active.

Batching and rate limiting. Requests are issued in parallel batches of 35 (Azure pools, across multiple endpoints) or 10 (Together AI, OpenRouter, single endpoint), with a 1-second inter-batch delay. Token usage is tracked per endpoint to stay within rate limits (250,000 tokens/minute per Azure resource); the runner rotates to the least-loaded endpoint within each pool.

Error handling and retries. Transient errors (HTTP 5xx, connection resets) trigger rotation across endpoints without delay; if all endpoints fail, exponential backoff

begins at 2 seconds. Content-filter rejections (HTTP 400) are retried once on each available endpoint; on OpenRouter, items rejected by Alibaba’s filter are automatically retried on an alternative provider (AtlasCloud) without logprobs. After 20 consecutive batch failures, execution pauses for 10 minutes before resuming. Maximum retries per item: 5 or the pool size, whichever is larger.

Output format. Each classified item is written as a single JSON line to a task-specific JSONL file, recording: the item ID, dataset, prompt variant, model output, timestamp, token counts, logprobs (when available), and an error flag. Files are append-only and resumable — restarting the runner skips items already present in the output file.

I. Cost and Latency

Table 29 compares the cost of reasoning and examples for GPT 5.4 at Azure pricing (\$2.25/M input, \$13.00/M output). Reasoning tokens are charged at the output rate ($5.78\times$ more expensive than input tokens), while examples add only input-rate tokens. The “Ratio” column expresses each intervention’s cost as a fraction of the average 0-shot prompt cost (\$3.83 per 1,000 calls, averaging over $P=548$, $L_I=2,214$, $L_{II}=2,346$ input tokens).

	μ_{tokens}	Cost	Ratio
<i>Reasoning</i>			
Low	113	1.47	$0.38\times$
Medium	322	4.19	$1.09\times$
High	746	9.70	$2.53\times$
Extra-high	2,479	32.23	$8.42\times$
<i>Examples</i>			
P examples	746	1.68	$0.44\times$
L_I examples	599	1.35	$0.35\times$
L_{II} examples	521	1.17	$0.31\times$

Note: GPT 5.4 Azure pricing as of April 2026: \$2.25/M input tokens, \$13.00/M output tokens. Cost in USD per 1,000 calls. Reasoning tokens are charged at the output rate ($5.78\times$ the input rate); examples are charged at the input rate. Ratio = intervention cost / average 0-shot prompt cost (3.83 per 1,000 calls). *Low* reasoning and adding examples cost a similar fraction of the prompt ($\approx 0.3\text{--}0.4\times$), but examples deliver far larger accuracy gains. *High* reasoning costs $2.5\times$ the entire prompt; *extra-high* costs $8.4\times$.

Table 29: Cost comparison: reasoning vs. examples (GPT 5.4).

Table 30 reports the latency cost of reasoning. For GPT 5.4, *low* reasoning doubles

median response time (1.4 s $\rightarrow$ 3.5 s); `high` is $8\times$ slower (11.6 s); `extra-high` is $21\times$ slower (30.2 s). Qwen reasoning is far more expensive in time: $50\text{--}58\times$ slower (median), reflecting less optimised reasoning infrastructure. Adding examples, by contrast, adds zero extra latency — input tokens are processed in parallel.

	Reasoning	Median	Slowdown
GPT 5.4	off	1.4	$1.0\times$
	low	3.5	$2.5\times$
	medium	6.8	$4.7\times$
	high	11.6	$8.1\times$
	extra-high	30.2	$21.0\times$
GPT 5.4-nano	off	1.0	$1.0\times$
	low	1.3	$1.3\times$
	high	1.9	$1.9\times$
	extra-high	5.4	$5.5\times$
Qwen 397B	off	0.8	$1.0\times$
	on	49.0	$58.6\times$
Qwen 9B	off	0.6	$1.0\times$
	on	38.3	$69.5\times$

Note: Median end-to-end response time per API call in seconds. Slowdown relative to no-reasoning baseline for each model. At 2,101 items, `extra-high` reasoning on GPT 5.4 takes ≈ 17.5 hours vs. ≈ 50 minutes without reasoning.

Table 30: Reasoning latency by model and effort level.

Adding examples costs roughly the same fraction of the prompt as `low` reasoning (both $\approx 37\text{--}38\%$) but delivers far larger accuracy gains — for example, L_{II} minimal accuracy rises from 19% to 60% with examples (Section 6.3), whereas reasoning at any level produces at most single-digit improvements on the same task (Section 6.4). The practical recommendation is to invest in prompt content (examples, detailed instructions) before enabling reasoning.

J. Codebook-to-Prompt Pipeline: Prompts

All LLM-assisted pipeline transformations in this appendix — filtering, compression, and the verbose and noise expansions — were produced with Claude Opus 4.6 at temperature 0 via Azure AI Foundry, applied once per codebook section. The compression and filtering outputs were validated by checking that a list of task-specific key terms survived the transformation, regenerating otherwise.

J.1. Filtering Prompt

The following prompt was used with Claude Opus 4.6 to filter each raw codebook, removing procedural content while preserving all classification-relevant material.

```
# General Task
- Filter a human classification codebook to retain only content relevant for an LLM classification prompt.

# Instructions
- Read the codebook below and remove all content that is only relevant to human annotators working in a laboratory setting.
- REMOVE the following types of content:
  + Greetings, welcomes, and introductory pleasantries
  + Time estimates ('`the experiment will take about 20 minutes`')
  + Payment, remuneration, bonus, and reward details
  + Excel sheet, spreadsheet, or file handling instructions
  + Column descriptions, data entry instructions, formatting guidance
  + Instructions to work independently or not discuss with others
  + Instructions to review or double-check coding
  + References to instruction sheets, quizzes, or other documents
  + Meta-instructions ('`the next page describes...`', '`we now describe...`')
  + Timeout warnings and deadline-related instructions
  + Separator lines (dashes, horizontal rules)
  + '`Let's illustrate...`' or similar transitional phrases before examples
- KEEP the following types of content:
  + Experiment mechanics (what players do, payoffs, choices, communication)
  + Classification definitions (what each label means, conditions for each category)
  + Examples with their classifications and explanatory remarks
  + Any domain-specific concepts needed for classification
- Light reframing: where the original text addresses subjects in second person ('`You will...`'), reframe to third person ('`Participants will...`').
- Do NOT add any information not present in the original.
- Do NOT summarise or shorten the kept content.
- Preserve the section structure with clear headers.
```

Figure 18: Filtering prompt used in Step 1 of the pipeline.

J.2. Compression Prompt

The following prompt was applied iteratively to generate compressed variants. Each application halves the text: $V \rightarrow C1$ ($\approx 50\%$) and $C1 \rightarrow C2$ ($\approx 25\%$). A list of task-specific key terms was enforced to prevent loss of essential vocabulary during compression.


```

# General Task
- Create an expert-level summary of the following experiment
description. This text has already been filtered to contain only
classification-relevant content.

# Instructions
- Shorten the text below to approximately half of its current
length.
- ONLY REMOVE information. Do NOT add any information,
inferences, or clarifications not explicitly stated in the
original text.
- You MUST keep the following terms exactly as they appear:
{preserved.terms}
- Prioritize keeping (in this order):
  + What participants choose and what happens as a result (game
mechanics, payoffs)
  + How participants communicate (message protocol)
  + The number and roles of participants
  + The objective of the game
- When removing content, prefer dropping:
  + Derived or illustrative quantities
  + Elaborations that restate a mechanic already captured
  + Treatment-specific details (e.g., number of rounds, group
re-matching)
- Do not paraphrase unless necessary for coherence after removing
a sentence.
- Present the output as a list of statements. Each statement
should be at most 2 sentences long. Start each statement on a
new line with ``- ``.

```

Figure 19: Compression prompt used in Step 3 of the pipeline (experiment context version).

A parallel prompt was used for theory sections, prioritizing level/type definitions and inter-level reasoning relationships over formal notation and numerical parameters.

J.3. Verbose and Noise Prompts

To test whether models respond to informational content or merely to prompt length (design in Appendix F.5; results in Appendix E), each verbatim section was rewritten in two ways with Claude Opus 4.6. The *verbose* variant (V_{ex}) elaborates the existing content without adding new information (word count +50–275%); the length-matched *noise* variant (V_{noise}) instead pads the same section to the verbose length with topic-irrelevant filler (red-squirrel ecology) interspersed at the sentence level. The contrast isolates the effect of relevance from that of length. The two generation prompts are shown in Figures 20 and 21; both were applied per section (experiment context, theory, classification instructions), and all original wording, structure, and category definitions were preserved.

```

# General Task
- Expand the following classification prompt section, making its
existing content fuller and more explicit without adding any new
information.

# Instructions
- Increase the length of the text below by roughly 50--275%,
keeping its meaning unchanged.
- ONLY ELABORATE on information already present. Do NOT
introduce new rules, categories, criteria, or conceptual content
not stated in the original.
- Permitted expansions:
  + Provide more detail about information already stated
  + Restate key points in alternative phrasings
  + Make implicit criteria explicit and spell out boundary cases
already implied
- Do NOT change any classification definition, label, or decision
rule.
- Preserve the section structure, headers, and the order of the
level/category descriptions.
- Keep all task-specific key terms exactly as they appear:
{preserved.terms}

# Text to expand
{text}

```

Figure 20: Verbose-expansion prompt used with Claude Opus 4.6 to generate the V_{ex} variants.

```

# General Task
- Lengthen the following classification prompt by inserting
topic-irrelevant filler, without changing any of its original
content.

# Instructions
- Insert sentences of irrelevant factual content (ecology of the
red squirrel, Sciurus vulgaris) into the text below.
- Add filler until the total word count matches the verbose ( $V_{\text{ex}}$ )
variant of the same section.
- Intersperse the filler at the sentence level --- distribute
it between existing sentences and bullet points rather than
appending it in one block.
- Do NOT modify, remove, paraphrase, or reorder any of the
original text. Every original sentence, header, label, and
classification rule must remain exactly as written.
- The inserted content must be entirely unrelated to the
classification task (no strategic reasoning, games, voting, or
promises).

# Text to pad
{text}

```

Figure 21: Noise-injection prompt used with Claude Opus 4.6 to generate the length-matched V_{noise} control variants.

K. Prompt Template Rationale

Our prompt template (Figure 1) was designed following established guidelines for effective prompt design (Zhao et al., 2023; Ziems et al., 2024; White et al., 2023; Mishra et al., 2021b). Research has shown that using instructions written for human annotators directly as prompts leads to poor LLM performance (Efrat and Levy, 2020), whereas reframing them into structured templates substantially improves performance (Mishra et al., 2021b). Each component of the template is motivated by specific findings in the prompting literature:

- **General Task.** A high-level task specification using key terms (“classify”, “message”, “promise”) primes the model toward the intended output without specifying how to accomplish the task (Reynolds and McDonell, 2021; Mishra et al., 2021a; White et al., 2023).
- **Role Persona.** Assigning a domain-specific persona (“behavioural economist specialised in level- k reasoning”) acts as an augmented task specification, drawing on the model’s pre-trained knowledge of expert reasoning patterns (Reynolds and McDonell, 2021; Kong et al., 2023). Persona assignment has been shown to elevate performance comparably to chain-of-thought prompting, though it may surface biases from training data (Salewski et al., 2024).
- **Context.** Background information about the experiment and, for L -tasks, the underlying theoretical framework. Previous studies document positive effects from including background details (Chae and Davidson, 2025; Savelka et al., 2023; Yuan et al., 2023; Clavié et al., 2023). In our pipeline, the verbatim codebook text is retained as the baseline and then systematically compressed to test how much detail is needed (Section 5.1).
- **Classification Task.** Expert-crafted category definitions from the original codebook, restructured into itemised lists. Expert label descriptions significantly outperform both no descriptions and automatically generated ones (Pesquine et al., 2023), and expert task descriptions surpass basic NLP-style instructions (Mishra et al., 2021a; Logan IV et al., 2021; Schick and Schütze, 2020).
- **Examples.** Worked examples from the original codebook, included verbatim where available. Included as a dedicated section to allow independent variation in the factorial design.

- **Constraints and Output Format.** Enforce structured JSON output for consistent, machine-parseable results (Reynolds and McDonell, 2021; Mishra et al., 2021b; White et al., 2023). Tam et al. (2024) find that format restrictions can degrade reasoning performance but actually improve accuracy on classification tasks, supporting the use of constrained JSON output in our setting.

Segmenting instructions into itemised lists improves model comprehension (Mishra et al., 2021b). We use Markdown formatting throughout, as it supports itemisation and subtitles with minimal token overhead. He et al. (2024) compare plain text, Markdown, JSON, and YAML formats across GPT-3.5 and GPT-4, finding that GPT-3.5 performance varies by up to 40% across formats while GPT-4 is substantially more robust — consistent with our finding that format sensitivity diminishes with model scale.

L. Prior Work Comparison

Table 31 compares our study with the most closely related prior work on prompt design for LLM text classification. Past work on prompt design has explored subsets of the space we cover — specifically content detail, surface variation, and model scale (see Table 31). When placed onto this three-dimensional map, their isolated findings align perfectly with our unified results.

Studies varying content detail (Lupo et al., 2024; Halterman and Keith, 2025; Ruckdeschel, 2025) recover the same patterns we report: different types of information can substitute for one another, and excessive detail overloads smaller models. Conversely, studies exploring surface variation — such as formatting, framing, and phrasing — (Reich et al., 2025; McLaren et al., 2026) find that these choices are largely inert when content is well-specified, but cause erratic behavior in smaller models. Atreja et al. (2025) varies both, noting that detailed definitions aid ambiguous tasks while surface variations shift the output distribution.

None of these studies, however, cross all three axes. Crucially, none test how surface variation and prompt detail interact at frontier model scale — which is the exact gap our design fills. We take each study in turn.

Study	Dimensions varied	Models	Scale	Tasks (labels)	Content ^a	Prompt ^b	Key finding
Lupo et al. (2024)	Description depth (3) × examples (0/15)	5	13B–GPT-4	Fatherhood roles (6)	Yes	~40–1,200	Examples dominant; descriptions partially substitute but add less when examples present
Halterman and Keith (2025)	6 per-label fields ^c (sequential removal)	4	7–12B	Political events (8–142)	Partial	~600–3,900	Small models rely on label names over definitions; zero-shot only
Ruckdeschel (2025)	Codebook none/short/full × examples (0/1/2)	3	7–27B	Gun violence frames (9–13)	Yes	~130–1,150	Codebook helps larger model; examples on top of codebook neutral or hurt
Reich et al. (2025)	Persona, context, coding strategy, reasoning, justification (864 combos) ^d	2	7–12B	Climate coding (2)	Partial	~100–200	Coding strategy dominant; persona and context not significant
Atreja et al. (2025)	Definition × output format × explanation × length (2 ⁴)	4	7B–GPT-4o	Toxicity, sentiment, stance, frames (2–9)	Yes	~25–325	Definitions help on ambiguous tasks; explanation shifts label distributions
McLaren et al. (2026)	Persona × reasoning × examples (0/ <i>n</i>)	6	~20–72B	Approval, sentiment, distance, topics (2–8)	No	~80–900	No single approach uniformly best; persona and reasoning erratic across models
This study	Content × format × framing × examples × reasoning	4	9B–GPT 5.4	Promise, level-<i>k</i> reasoning (2–6)	Yes	~110–1,500	Content dominant; surface irrelevant on rich prompts; scale moderates all dimensions

^aContent richness varied independently of other dimensions. “Partial” = content varied but compensation effects not testable. ^bApproximate word count of the prompt (excluding the item text), from the smallest to the richest variant tested. ^cLabel name, definition, clarification, negative clarification, positive example, negative example. ^d8 dimensions with 2–3 levels each: persona, context, task specification, coding strategy, coding elements, reasoning trigger, reasoning steps, justification.

Table 31: Comparison with prior work on prompt design for LLM text classification.

Lupo et al. (2024) provide the clearest prior evidence for information substitutability: adding category descriptions helps most when examples are absent, and vice versa. We extend this by decomposing content into four functional components (experiment context, theory, classification instructions, examples) at multiple compression levels rather than a binary presence/absence, by crossing content with formatting, framing, and reasoning, and by testing across a wider model range including frontier models. Their prompts range from ~ 40 to $\sim 1,200$ words, comparable to ours when examples are included, while focusing on two dimensions (description depth and shot count).

Halterman and Keith (2025) ablate six per-label codebook fields and find that small models fall back on label name heuristics when definitions are removed. This is consistent with our finding that smaller models are more dependent on detailed classification instructions. We extend their work by testing larger models (up to GPT 5.4), where this dependence diminishes, and by varying content richness independently of other prompt dimensions rather than in sequential removal. Their prompts reach $\sim 3,900$ words for the 142-label manifesto task, the largest in this literature; their study focuses on two smaller models (7–12B), leaving open the question of whether frontier models would show the same sensitivity.

Ruckdeschel (2025) find that providing the full codebook helps the largest model in their study (Gemma 27B) but that adding examples on top of the codebook is neutral or harmful. This parallels our finding that examples compensate for missing content but add little when the codebook is already rich. The scale interaction they observe — the larger model benefits from detail while the smaller ones are confused by it — matches ours in direction, though it is confined to small-to-mid open models (7–27B) and does not reach frontier scale. We complement their coarse codebook contrast (none/short-/full) by testing finer graded compression levels, and extend the model range up to GPT 5.4.

Reich et al. (2025) systematically vary eight prompt engineering dimensions and find that the coding strategy (i.e., how detailed the classification instructions are) is the dominant factor — consistent with our finding that classification instructions account for the majority of explained variance. Their finding that persona and context are not significant mirrors our framing results on well-specified prompts. We extend by independently varying content richness, which their design holds approximately constant within a compact ~ 100 – 200 word prompt range, and by testing whether their conclusions hold at larger model scales.

Atreja et al. (2025) use a full 2^4 factorial on four prompt features and find that category definitions help on ambiguous multi-class tasks but not on clear binary ones.

This parallels our distinction between learning-heavy tasks (where detailed instructions are essential) and recognition-heavy tasks (where model choice matters more than content). They explicitly call for future work testing interactions between prompt dimensions; our factorial design with interaction analysis addresses this directly. Their prompts span ~ 25 – 325 words, placing them near the lower end of the specification range where our results would predict sensitivity to surface variations to be highest.

McLaren et al. (2026) hold codebook content fixed — and relatively rich, with label definitions, decision rules, and worked examples — and vary only model choice, persona, reasoning, and examples across six open-weight models (~ 20 – 72 B, quantised). Their headline finding — that interaction effects dominate main effects and no single approach is uniformly best — shows that surface variations can remain erratic even on richer content when the models are small open-weight ones, consistent with our finding that smaller models retain a surface sensitivity that frontier models do not. We extend by varying content richness directly and by testing frontier-scale proprietary models, where this inconsistency largely disappears — a comparison their fixed-content, open-weight design cannot make. Their prompt range (~ 80 – 900 words) is moderate, and their focus on open-weight models at quantised precision leaves open the question of how the same patterns extend to frontier-scale proprietary models.

M. F1 Metric Replication

The main text reports accuracy throughout. This appendix replicates the headline tables and information surfaces using F1, to verify that the qualitative findings do not depend on the choice of scoring metric.

Metric. We report support-weighted macro F1: per-class F1 averaged with weights equal to each class’s support (the number of items where that class is a valid answer). Micro F1 equals accuracy by definition for single-label tasks and is uninformative here; unweighted macro F1 over-penalises rare levels, which is inappropriate because strategic-reasoning level distributions are systematically skewed toward low levels (Camerer et al., 2004; Crawford et al., 2013) and levels 4–5 appear in only a handful of L_{II} items. Weighted F1 mirrors accuracy’s implicit frequency weighting while still decomposing per-class precision and recall. For L_{II} we preserve the interval-membership agreement rule used throughout the main text: $TP(c)$ = prediction = c with c in the interval; $FP(c)$ = prediction = c with c outside the interval; $FN(c)$ is counted only when the prediction itself falls outside the interval, so picking one valid level over another does not generate an FN.

Confidence intervals. Per-cell F1 values carry 95% item-level bootstrap intervals ($B = 1,000$, seed 42). F1 deltas carry paired bootstrap intervals: a single resample of item indices is applied to both conditions before the delta is computed, yielding tighter within-model intervals.

M.1. F1 at minimal vs verbatim content

	Minimal			Verbatim		
	P	L_I	L_{II}	P	L_I	L_{II}
GPT 5.4	.73 [.71, .75]	.47 [.42, .51]	.51 [.48, .55]	.74 [.71, .76]	.81 [.77, .84]	.78 [.75, .81]
GPT 5.4-nano	.60 [.57, .62]	.50 [.46, .54]	.53 [.50, .56]	.64 [.61, .66]	.68 [.64, .72]	.65 [.62, .69]
Qwen 397B	.68 [.66, .71]	.50 [.46, .54]	.42 [.38, .45]	.62 [.60, .65]	.82 [.79, .86]	.73 [.70, .76]
Qwen 9B	.62 [.60, .65]	.53 [.49, .57]	.40 [.37, .43]	.62 [.59, .65]	.71 [.67, .75]	.56 [.52, .61]

Note: Support-weighted macro F1 for all three tasks with 95% item-level bootstrap CI ($B = 1,000$) in brackets. L_{II} uses the interval-membership agreement rule from the main text. Compare to Table 3.

Table 32: F1 at the minimal and verbatim prompts.

M.2. F1 with the full codebook

	P	L_I	L_{II}
GPT 5.4	.88 [.86, .90]	.84 [.80, .87]	.78 [.75, .81]
GPT 5.4-nano	.72 [.69, .74]	.77 [.73, .81]	.65 [.61, .69]
Qwen 397B	.87 [.85, .89]	.83 [.80, .86]	.75 [.72, .78]
Qwen 9B	.78 [.77, .80]	.82 [.79, .86]	.61 [.57, .65]

Note: Support-weighted macro F1 on verbatim content + n -shot examples, no reasoning, with 95% item-level bootstrap CI. Compare to Table 6.

Table 33: F1 with the full codebook.

M.3. F1 deltas: examples and content

Tables 34–37 replicate the examples-vs-content decomposition (main-text Table 5 and Appendix C.2) under weighted F1 with paired bootstrap CIs.

	P	L_I	L_{II}
Δ_{Ex}	+17 [+.14, +.19]	+38 [+.33, +.44]	+17 [+.14, +.22]
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+16 [+.14, +.18]	+04 [+.01, +.08]	-.09 [-.13, -.05]
$\Delta_{\text{Ex} \text{Content}}$	+14 [+.12, +.17]	+03 [+.01, +.06]	-.00 [-.02, +.02]
$\Delta_{\text{Content} \text{Ex}}$	-.02 [-.03, -.01]	-.01 [-.04, +.01]	+.09 [+.05, +.13]

Note: F1 deltas with 95% paired bootstrap CI ($B = 1,000$). A CI excluding zero indicates a directionally reliable effect. Compare to Table 5.

Table 34: GPT 5.4: effect of examples (F1 metric).

	P	L_I	L_{II}
Δ_{Ex}	+09 [+.06, +.12]	+22 [+.17, +.27]	+05 [+.01, +.09]
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+05 [+.03, +.08]	+04 [-.00, +.09]	-.07 [-.11, -.03]
$\Delta_{\text{Ex} \text{Content}}$	+08 [+.05, +.10]	+09 [+.06, +.13]	-.00 [-.03, +.02]
$\Delta_{\text{Content} \text{Ex}}$	+03 [+.01, +.05]	+05 [+.01, +.09]	+07 [+.03, +.11]

Table 35: GPT 5.4-nano: effect of examples (F1 metric).

	P	L_I	L_{II}
Δ_{Ex}	+19 [+.16, +.21]	+33 [+.27, +.37]	+26 [+.22, +.30]
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+24 [+.21, +.27]	+00 [-.03, +.03]	-.05 [-.09, -.02]
$\Delta_{\text{Ex} \text{Content}}$	+24 [+.21, +.27]	+01 [-.02, +.03]	+02 [-.00, +.04]
$\Delta_{\text{Content} \text{Ex}}$	-.00 [-.01, +.01]	+01 [-.02, +.04]	+08 [+.04, +.10]

Table 36: Qwen 397B: effect of examples (F1 metric).

	P	L_I	L_{II}
Δ_{Ex}	+16 [+.13, +.19]	+13 [+.07, +.18]	+26 [+.22, +.31]
$\Delta_{\text{Ex}} - \Delta_{\text{Content}}$	+16 [+.14, +.19]	-.05 [-.10, -.01]	+09 [+.06, +.13]
$\Delta_{\text{Ex} \text{Content}}$	+16 [+.14, +.19]	+12 [+.08, +.15]	+05 [+.02, +.08]
$\Delta_{\text{Content} \text{Ex}}$	+00 [-.01, +.02]	+17 [+.13, +.21]	-.05 [-.08, -.01]

Table 37: Qwen 9B: effect of examples (F1 metric).

M.4. F1 deltas: content vs reasoning

Tables 38–41 replicate the content-vs-reasoning decomposition (main-text Table 7 and Appendix D.3) under weighted F1 with paired bootstrap CIs.

	P	L_I	L_{II}
Δ_{Rea}	+.09 [+.07, +.12]	-.16 [-.21, -.11]	+.17 [+.13, +.21]
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+.09 [+.07, +.11]	-.50 [-.55, -.44]	-.10 [-.14, -.06]
$\Delta_{\text{Content} \text{Rea}}$	-.12 [-.14, -.09]	+.51 [+.45, +.57]	+.10 [+.06, +.14]
$\Delta_{\text{Rea} \text{Content}}$	-.03 [-.05, -.01]	+.02 [-.01, +.04]	+.00 [-.03, +.03]

Note: F1 deltas with 95% paired bootstrap CI. GPT uses high reasoning effort; Qwen uses reasoning: on. Compare to Table 7.

Table 38: GPT 5.4: content vs. reasoning substitutability (F1 metric).

	P	L_I	L_{II}
Δ_{Rea}	+.10 [+.07, +.12]	+.13 [+.08, +.18]	-.09 [-.13, -.06]
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+.05 [+.03, +.08]	-.05 [-.10, +.00]	-.22 [-.26, -.17]
$\Delta_{\text{Content} \text{Rea}}$	-.00 [-.02, +.02]	+.12 [+.07, +.17]	+.21 [+.15, +.26]
$\Delta_{\text{Rea} \text{Content}}$	+.05 [+.03, +.07]	+.07 [+.03, +.12]	-.01 [-.04, +.03]

Table 39: GPT 5.4-nano: content vs. reasoning substitutability (F1 metric).

	P	L_I	L_{II}
Δ_{Rea}	+.00 [-.02, +.02]	+.05 [+.01, +.10]	-.02 [-.05, +.01]
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+.06 [+.04, +.08]	-.28 [-.33, -.23]	-.33 [-.37, -.29]
$\Delta_{\text{Content} \text{Rea}}$	+.05 [+.03, +.07]	+.28 [+.24, +.33]	+.33 [+.28, +.37]
$\Delta_{\text{Rea} \text{Content}}$	+.10 [+.08, +.13]	+.01 [-.02, +.03]	-.00 [-.04, +.03]

Table 40: Qwen 397B: content vs. reasoning substitutability (F1 metric).

	P	L_I	L_{II}
Δ_{Rea}	+.04 [+.02, +.06]	+.07 [+.01, +.11]	-.11 [-.15, -.08]
$\Delta_{\text{Rea}} - \Delta_{\text{Content}}$	+.04 [+.02, +.06]	-.12 [-.17, -.06]	-.28 [-.33, -.24]
$\Delta_{\text{Content} \text{Rea}}$	+.06 [+.04, +.09]	+.21 [+.16, +.26]	+.35 [+.31, +.40]
$\Delta_{\text{Rea} \text{Content}}$	+.10 [+.08, +.13]	+.10 [+.05, +.14]	+.07 [+.03, +.11]

Table 41: Qwen 9B: content vs. reasoning substitutability (F1 metric).

M.5. F1 deltas: reasoning effect on verbatim content (all models)

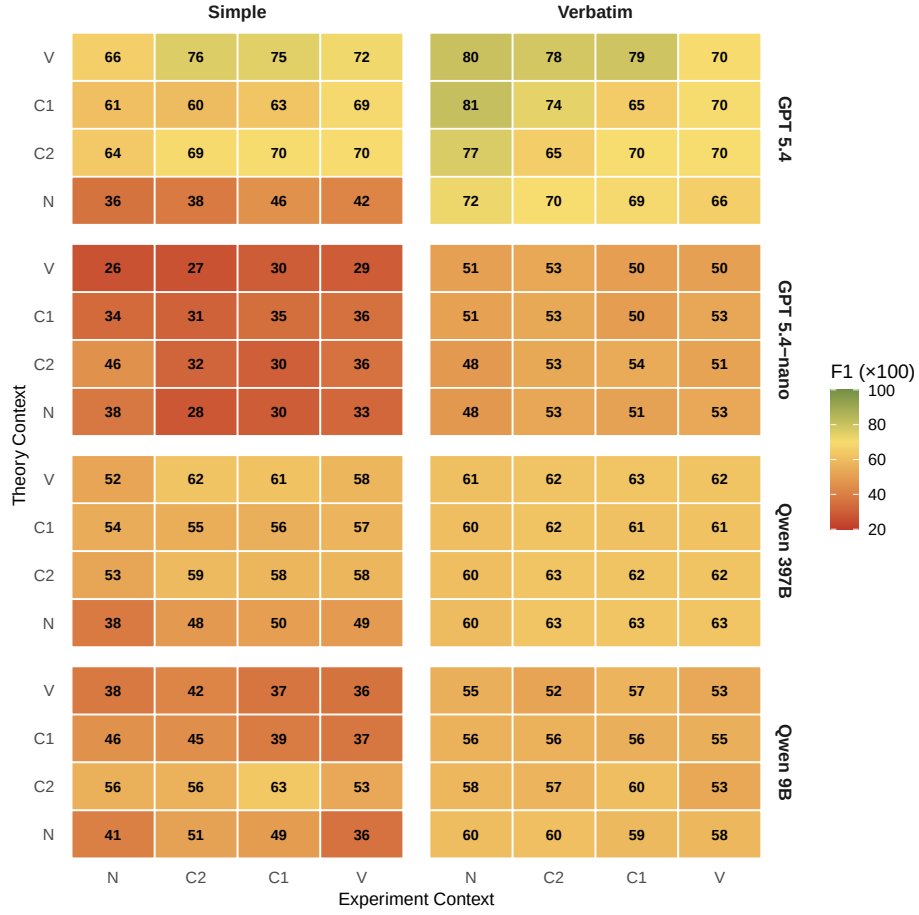
	0-shot			n -shot		
	P	L_I	L_{II}	P	L_I	L_{II}
GPT 5.4	-.03 [-.05, -.01]	+.02 [-.01, +.04]	+.00 [-.03, +.03]	+.01 [+.00, +.03]	+.02 [-.00, +.04]	+.03 [-.00, +.06]
GPT 5.4-nano	+.05 [+.02, +.08]	+.07 [+.03, +.12]	-.01 [-.04, +.03]	+.11 [+.09, +.13]	+.02 [-.02, +.05]	-.02 [-.05, +.01]
Qwen 397B	+.10 [+.08, +.13]	+.01 [-.01, +.03]	-.00 [-.03, +.03]	+.03 [+.02, +.05]	+.03 [+.01, +.05]	+.02 [+.00, +.05]
Qwen 9B	+.10 [+.08, +.13]	+.10 [+.06, +.14]	+.07 [+.03, +.11]	+.09 [+.07, +.11]	+.02 [-.01, +.04]	+.07 [+.03, +.11]

Note: F1 deltas (verbatim + reasoning minus verbatim) with 95% paired bootstrap CI. GPT models use high reasoning effort; Qwen models use reasoning: on. Compare to Table 22.

Table 42: Effect of reasoning on verbatim content (F1 metric).

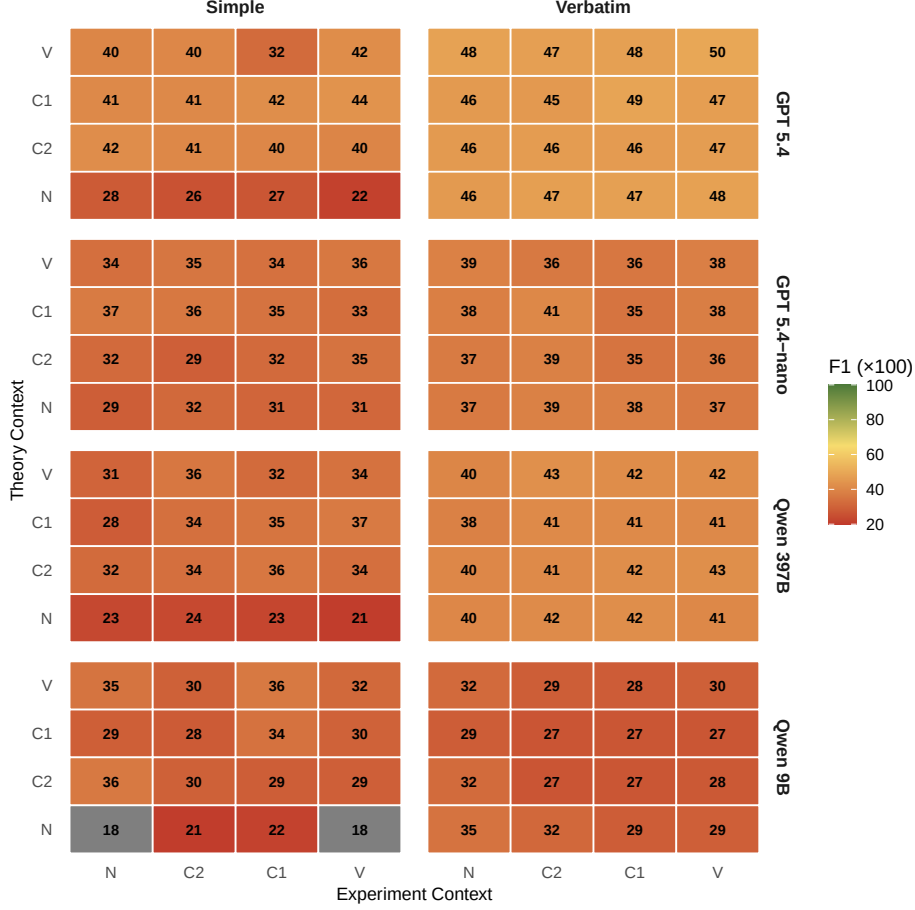
M.6. F1 information surfaces

Figures 22 and 23 replicate the L_I and L_{II} information surfaces (Section 6.1) under F1 scoring for all four models.



Note: F1 ($\times 100$) by experiment context (x-axis) and theory context (y-axis), separately under simple and verbatim classification instructions, for all four models. Compare to Figure 2.

Figure 22: L_1 information surface (F1 metric).



Note: F1 ($\tilde{A}100$) by experiment context and theory context for L_{II} , all four models. Compare to the L_{II} surface in Appendix A.1.

Figure 23: L_{II} information surface (F1 metric).

M.7. Patterns and comparison with accuracy

Weighted F1 tracks the main-text accuracy numbers closely — same agreement protocol, same class weighting — so no bold ranking reverses and no main-text claim is overturned. Four patterns stand out across the eleven F1 tables.

1. *Examples universally help.* Δ_{Ex} is positive with a CI that excludes zero in all 12 model $\times$ task cells: $+.09$ to $+.19$ on P , $+.13$ to $+.38$ on L_I , $+.05$ to $+.26$ on L_{II} . The weakest effect is GPT 5.4-nano on L_{II} ($+.05$), still a real gain.

2. *Content-vs-examples substitution scales with the task and the model.* On P , $\Delta_{Content|Ex}$ is essentially zero for every model: once examples are present, adding verbatim content adds nothing for the recognition-heavy task. On L_{II} , content still adds $+.07$ to $+.09$ for GPT 5.4, GPT 5.4-nano, and Qwen 397B even when examples are already present. Two model-specific exceptions stand out. Qwen 9B on L_I has

$\Delta_{\text{Content|Ex}} = +.17 [+.13, +.21]$ — the smallest model needs both full content and examples to reach its .82 ceiling. Qwen 9B on L_{II} has $\Delta_{\text{Content|Ex}} = -.05 [-.08, -.01]$ — the only cell where adding content on top of examples significantly hurts, suggesting the small model is saturated.

3. *Reasoning cannot substitute for content, and its value drops with scale.* $\Delta_{\text{Rea|Content}}$ is essentially zero for GPT 5.4 on every task, but becomes larger and reliably positive for smaller models on P (+.05 for nano, +.10 for Qwen 397B, +.10 for Qwen 9B). Qwen 9B is unique: reasoning-on-content is significant on all three tasks (+.10, +.10, +.07), meaning even with a full codebook and examples, the smallest model still extracts measurable benefit from reasoning.

4. *Reasoning without content can actively hurt.* Δ_{Rea} (reasoning added to the minimal prompt) is reliably negative on L_{I} for GPT 5.4 (−.16 [−.21, −.11]) and on L_{II} for every model except GPT 5.4 (nano −.09, Qwen 397B −.02 n.s., Qwen 9B −.11). Without classification instructions, reasoning leads the model toward plausible but wrong categorisations rather than correct ones.

Beyond these patterns, two observations about the metric itself are worth noting. On L_{I} , weighted F1 for frontier models tracks accuracy within a point (GPT 5.4: .84 vs. 84.0%; Qwen 397B: .83 vs. 83.6%), but for Qwen 9B it is several points below (.82 F1 vs. 82.8% accuracy), confirming that the smaller model concentrates its errors on minority levels. On L_{II} , weighted F1 is systematically a few points below interval accuracy for every model (GPT 5.4: .78 vs. 81.7%; Qwen 9B: .61 vs. 68.2%), reflecting uneven per-level recall: models often predict the same level within the interval and under-cover others. For aggregate agreement with human labels, interval accuracy is equivalent; for downstream analyses that depend on per-level reliability, weighted F1 is the more diagnostic metric.